

Neuro — 最终项目构思（PAD 产品架构文档）

版本说明： 本文档以 1.0 手写项目构思为底稿，融合 2.0 GPT 深度整理、3.0 互联网大厂产品方法论、4.0 产品文档体系规范，以及 2026-07-05 最新讨论中的 Neuron 模型、Knowledge Space、物理/逻辑分离、四层架构 等关键升级。版本优先级：最新讨论 > 4.0 > 3.0 > 2.0 > 1.0，冲突处以时间更新的内容为准。

v2.1 更新（2026-07-05 架构评审）： 将 `knowledgePaths` 重构为 `knowledgeParents`（存父 Space ID 列表，DFS 构建树）；`physicalPath` 改为 Project Relative Path；新增 `kind`（子类型）、`schemaVersion`、`capabilities`（Manifest）、Space `behavior`（Navigation/KnowledgeAggregation）字段；graph.db 明确只存 Entity Relation，不存 Navigation。

v2.2 更新（2026-07-05 架构补充）： 新增「Metadata 判定原则」：可作为 View 筛选条件的数据一律属于 metadata；新增「Event System & Transaction」（1.10 节）：事件驱动的事务架构确保跨层操作一致性；Knowledge.md 重构为统一五章结构（总结 / 为什么 / 怎么做 / 思考 / 经验），metadata 字段（Title/Tags/Status 等）移出 Knowledge.md；.neuro/persist/ 新增 event_log.db（事务崩溃恢复日志）。

第一部分：产品愿景（Vision）

为什么世界需要 Neuro？

当前知识管理工具存在根本性分裂：

- **Zotero / Mendeley** — 擅长文献管理，但缺乏 AI 分析与知识关联
- **Obsidian / Notion** — 擅长笔记与知识链接，但缺乏自动化的信息采集与 AI 驱动
- **Trello / Asana** — 擅长项目管理，但缺乏知识沉淀与智能分析
- **AI Chat（ChatGPT / Claude）** — 擅长对话与生成，但缺乏结构化的知识管理与持久化

而即便将 VSCode + Zotero + Obsidian + Claude Code 组合使用，也只能实现 Neuro 目标的 **60%~75%**，缺失的核心 25%~40% 包括：

- **Neuron**（不是一个文件，而是一个知识对象）
- **Knowledge.md**（AI 持续维护的活知识索引）
- **Knowledge Growth**（知识持续成长）
- **Knowledge Cultivation**（AI 主动培育知识，而非被动回答）
- **Project Memory**（持久化的知识数据库，跨会话保留）

Neuro 的目标： 打破工具壁垒，创建一个 **AI Native Knowledge Operating System**（AI 原生知识操作系统）。

产品定位

Neuro 不是文献管理软件。
不是 AI Chat。
不是 Obsidian。
不是项目管理工具。

Neuro 是 AI Research IDE。

产品哲学

Everything is a Neuron.
Everything has AI.
Everything is Connected.
Everything can Grow.
Everything can Evolve.

- 一切皆神经元
- 一切皆 AI
- 一切皆关联
- 一切皆可成长
- 一切皆可演化

Neuro 不是一个知识管理系统（Knowledge Management System）。

Neuro 是一个知识培育系统（Knowledge Cultivation System）。

也是 AI 原生的知识运行时（Knowledge Runtime）。

它不是用来"保存"知识的，而是用来"培养"知识的。

核心竞争力

Neuro 的不可替代性不在于任何一个单项能力，而在于以下能力的系统性组合：

能力	单项拆开看	组合起来看
Neuron 模型	"不就是对象嘛"	统一所有知识类型（论文/任务/实验/会议...）使用同一数据结构
物理/逻辑分离	"不就是标签系统"	knowledgeParents 实现一个 Neuron 同时在多个逻辑路径存在，物理存储只有一份
Knowledge.md	"不就是 AI 写摘要"	AI 持续维护的活知识索引 + 人机协同编辑
知识培育（Knowledge Cultivation）	"不就是定时提醒"	事件驱动的任务管理系统：AI 引导创建项目计划 → 甘特图可视化 → 拖拽编辑实时同步 → AI 事件驱动跟进 → 桌面 AI 助手辅助
四层架构 + Runtime	"不就是数据库"	Fact + Knowledge + Relation + Semantic 四层分离，各自独立优化
统一视图系统	"不就是多种展示方式"	Markdown / Tree / Graph / Timeline / Kanban / Calendar 全部统一为 View

Neuro 真正的护城河不是任何一个功能，而是这些功能组合在一起形成的系统性体验——从 Connector 采集 → Neuron 创建 → AI 理解与成长 → Relation 建立 → 可视化工作 → 知识培育演化的完整闭环。

而其中**任务管理系统（知识培育 + 可视化操作 + AI 事件驱动跟进）**是用户最能直接感知到的核心竞争力，也是区别于 Obsidian / Notion / Zotero 的最强差异点。

第二部分：产品架构文档（PAD）

本文档是 Neuro 的 "宪法"。所有功能开发、UI 设计、技术选型，均以此处定义的概念为准。

违反以下五条原则的功能，直接删除：

1. Everything is a Neuron
2. Everything has AI
3. Everything is Connected
4. Everything can Grow
5. Everything can Evolve

第一章 核心概念（Core Concepts）

1.1 Neuron（神经元）—— 第一性原则

Everything is a Neuron.

这是 Neuro 的第一性原则（First Principle）。

项目名 **Neuro** 取义于"神经元（Neuron）"。正如大脑由神经元构成，Neuro 中的所有知识对象都是 **Neuron**。

不同 Neuron 的职能不同，但数据结构完全统一：

```
Neuron
├── metadata.json  ← 事实层（Fact Layer）— 它是什么
├── Knowledge.md   ← 知识层（Knowledge Layer）— 它意味着什么
└── Assets/        ← 资源层（Asset Layer）— 附件文件
```

设计决定： 每个 Neuron 内部不再有 `.runtime/` 目录。所有运行时数据（关系数据库、向量索引、缓存等）统一由项目级的 `.neuro/` 管理，避免双重结构带来的混乱。参见 1.6 节。

Neuron ID 命名规则

设计原则：ID 应有意义。

- **默认命名：** 反映 Neuron 的类型和内容特征，如 `Paper_Flexible_LIG_Electrode_2025`、`Task_Make_LIG_Electrode`、`Space_Brain_Organoid`
- **允许用户自定义：** 用户可随时重命名 Neuron 目录，`metadata.json` 中的 `id` 字段同步更新
- **路径稳定性：** Neuron Tree 基于 `knowledgeParents`（父 Space ID）DFS 构建，不依赖目录名，因此重命名不会导致导航路径断裂
- **唯一性保证：** 同一 Project 内 ID 唯一，重命名时检测冲突

核心观点： Knowledge Space 和 Knowledge Unit 不是不同的东西——它们都是 Neuron，只是 `type` 不同。

1.2 Neuron 类型体系

Type 值	角色	metadata.json 关键字段	Knowledge.md 内容
<code>project</code>	项目根节点	<code>type</code> , <code>status</code> , <code>members</code>	项目概述、目标、状态、关键数据

Type 值	角色	metadata.json 关键字段	Knowledge.md 内容
space	知识空间 (分类/集合节点)	type, kind, behavior, cover	该领域综述、统计、研究热点、AI 自动总结
paper	论文/文献	type, kind, authors, doi, journal, year, language	摘要、关键参数、创新点、AI 分析
task	任务	type, kind, assignee, startDate, endDate (均含具体时间, ISO 8601) , priority, status, progress	目标、当前进度、AI 建议、风险
experiment	实验	type, kind, method, date, status	实验设计、参数、结果、分析
meeting	会议	type, kind, participants, date	议程、要点、待办
idea	想法/灵感	type, kind, source	描述、背景、相关参考
person	人员/合作者	type, name, affiliation, email, researchInterests	信息、研究方向、相关项目
group	课题组/实验室/机构	type, name, location(lat/lng), paperCount, citationCount	课题组概况、研究方向、成员、代表性成果
webpage	网页	type, kind, url, snapshot, source	摘要、核心观点、截图、相关链接
device	设备/仪器	type, kind, specs, location	规格、使用记录、维护日志
protocol	实验方案/标准流程	type, kind, steps, safety	步骤、参数、注意事项
dataset	数据集	type, kind, format, size, source	描述、结构、使用方法
note	自由笔记	type, kind, source	用户自由书写

注意： `webpage`（网页）是独立的 Neuron 类型，与 `group`（课题组）共享 Markdown 渲染视图，但 `metadata` 结构不同——`webpage` 记录 URL 和截图，`group` 记录地理位置和论文统计。两者的区别体现在 Fact Layer（`metadata.json`）而非 Knowledge Layer（`Knowledge.md`）。

kind（子类型）—— 在 type 模板内的细粒度分类

每个 Neuron 有 `type`（高维分类）和 `kind`（子类型）两层分类。两者的关系是：

`type` 决定 Neuron 的骨架（数据模型 + 模板结构），`kind` 决定这个骨架内的具体形态（预设字段、初始内容、AI 解析策略）。

类比到 Zotero 的文献导入：当 Connector 从网页抓取论文时，Zotero 会自动区分这是一篇博士论文、期刊研究文献、还是综述文章——三种情况共享"文献"这个 `type`，但 `metadata` 字段和导入表单不同（博士论文有答辩日期和学校，期刊文章有卷期号，综述有关键词总结）。

Neuro 中 `kind` 解决的正是类似问题：

Type	默认 kind	其他内置 kind 示例	区分意义
paper	research	review / patent / preprint / thesis	研究论文关注实验方法和数据；综述关注引用网络和趋势分析；专利关注权利要求和技术路线；学位论文关注学校和导师

Type	默认 kind	其他内置 kind 示例	区分意义
task	todo	bug / milestone / routine	软件默认只提供todo 作为默认 kind，其他由用户自定义
experiment	simulation	wetlab / animal / clinical	软件默认只提供simulation 作为默认 kind，其他由用户自定义
webpage	article	documentation / tutorial / news / blog	软件默认只提供article 作为默认 kind，其他由用户自定义
meeting	weekly	review / one-on-one / conference	软件默认只提供weekly 作为默认 kind，其他由用户自定义
idea	insight	hypothesis / brainstorm / question	软件默认只提供insight 作为默认 kind，其他由用户自定义
note	journal	log / sketch / summary	软件默认只提供journal 作为默认 kind，其他由用户自定义

Kind 默认策略：

- paper 类型因 Connector 导入时需自动区分文献子类型（如 Zotero 的行为），因此软件提供多组内置 kind
- 其他所有 type 的默认 kind 只有一个，其余内置 kind 作为"可选模板种子"展示给用户参考，但不会自动创建
- 用户可随时自定义新的 kind 值（如为 task 创建 grant_application、为 experiment 创建 field_trip），kind 值存储在 metadata.json 的 kind 字段中，无需修改软件代码

设计原则： type 决定 Neuron 的数据模型和模板骨架（Fact Layer 字段 + Knowledge.md 结构）， kind 决定该骨架内的具体预设和 AI 解析策略（哪些字段必填、AI 如何理解内容）。 type 由软件定义， kind 是用户可扩展的开放体系。

统一数据结构意味着： 无论未来增加什么类型，都不需要修改目录结构，只需新增 type 和对应模板即可。

1.3 metadata.json —— 事实层（Fact Layer）

职责： 回答"它是什么（What it is）"。

只存放结构化事实（客观数据），不存放 AI 推理或用户主观内容。

本章节以用户 Zotero 资料库中两篇实际论文作为贯穿示例，展示 metadata.json 如何映射真实元数据：

文献 A（研究论文， kind: research）：

- 标题： Stretchable Mesh Nanoelectronics for 3D Single-Cell Chronic Electrophysiology from Developing Brain Organoids
- 作者： Paul Le Floch, Qiang Li, Zuwan Lin 等（8 位）
- 期刊： Advanced Materials, 2022, 34(11), 2106829
- DOI： 10.1002/adma.202106829
- Zotero 标签： 共生电极、神经电极、精读

文献 B（研究论文， kind: research）：

- 标题： Rapid prototyping of a 3D well-shaped, porous, microelectrode array for extracellular recordings from cardiac cell layers and cortical organoids
- 作者： Zeynep Izlen Erenoglu, Lukas Hiendlmeier, Fulvia Del Duca 等（10 位）
- 期刊： Biofabrication, 2026, 18(2), 025005

- DOI: 10.1088/1758-5090/ae40a0
- Zotero 标签：袋式电极、神经电极、精读

以下所有 metadata.json 示例均基于这两篇真实文献的数据字段结构。

以下用完整的 **metadata.json** 展示文献 A（Le Floch 2022）从 Connector 导入后的数据，逐字段说明含义。注意 **physicalPath** 和 **knowledgeParents** 已在其中，其设计规则的详细说明见下方独立小节。

```
{
  "id": "Paper_LeFloch_Stretchable_Mesh_Nanoelectronics_2022",
  "type": "paper",
  "kind": "research",
  "schemaVersion": "1.0",

  "title": "Stretchable Mesh Nanoelectronics for 3D Single-Cell Chronic Electrophysiology from Developing Brain Organoids",
  "authors": [
    "Paul Le Floch",
    "Qiang Li",
    "Zuwan Lin",
    "Siyuan Zhao",
    "Ren Liu",
    "Kazi Tasnim",
    "Han Jiang",
    "Jia Liu"
  ],
  "doi": "10.1002/adma.202106829",
  "year": 2022,
  "journal": "Advanced Materials",
  "volume": "34",
  "issue": "11",
  "pages": "2106829",
  "issn": "1521-4095",
  "language": "en",
  "publisher": "Wiley Online Library",
  "url": "https://onlinelibrary.wiley.com/doi/abs/10.1002/adma.202106829",
  "keywords": "lefloch2022stretchable",

  "tags": ["共生电极", "神经电极", "精读"],
  "physicalPath": "研究生/文献库/脑类器官/脑类器官神经电极/",
  "knowledgeParents": ["Space_脑类器官神经电极"],

  "capabilities": [
    "Editable",
    "Embeddable",
    "AIUnderstandable",
    "Exportable",
    "Versioned"
  ],
  "createdAt": "2026-06-13T23:51:03Z",
  "updatedAt": "2026-07-04T17:05:42Z",
  "status": "reading"
}
```

关键字段说明：

字段	含义	来源	示例值
id	Neuron 唯一标识符，由类型 + 作者 + 缩写标题 + 年组成	自动生成，用户可修改	Paper_LeFloch_Stretchable_Mesh_Nanoelectronics_2022
type	高维分类，决定	Connector 根据条目	paper

字段	含义	来源	示例值
	Neuron 的模板骨 架	类型填充	
kind	子类型， 在 type 模板内细 分	Connector 判断（如 Zotero 区 分期刊文 章/学位论 文）	research — 研究论文
title	标题	Connector 从元数据 抓取	Stretchable Mesh Nanoelectronics…
authors	作者列表	Connector 从元数据 抓取	["Paul Le Floch", "Jia Liu", ...]
doi	数字对象 标识符	Connector 从元数据 抓取	10.1002/adma.202106829
journal	期刊名	Connector 从元数据 抓取	Advanced Materials
year	出版年份	Connector 从元数据 抓取	2022
volume / issue / pages	卷/期/页 码	Connector 从元数据 抓取	34 / 11 / 2106829
issn	国际标准 连续出版 物号	Connector 从元数据 抓取	1521-4095
language	语言	Connector 从元数据 抓取	en (ISO 639-1)
publisher	出版机构	Connector 从元数据 抓取	Wiley Online Library
url	来源 URL	Connector 从元数据 抓取	https://onlinelibrary.wiley.com/doi/abs/10.1002/adma.20210
keywords	引用关键 词 (Zotero 引用键)	Connector 从元数据 抓取	lefloch2022stretchable
tags	用户标签	Connector 导入时用	["共生电极", "神经电极", "精读"]

字段	含义	来源	示例值
户可添加			
physicalPath	项目相对路径，Neuron 在硬盘上的实际位置	Connector 导入时用户选择目录	研究生/文献库/脑类器官/脑类器官神经电极/
knowledgeParents	父 Space ID 列表，Neuron 在逻辑树中的位置	Connector 导入时用户选择目录	["Space_脑类器官神经电极"]
capabilities	能力清单，声明该 Neuron 支持的操作	根据 type + kind 自动计算	["Editable", "Embeddable", ...]
createdAt	创建时间（ISO 8601）	Connector 导入时记录	2026-06-13T23:51:03Z
updatedAt	最后修改时间（ISO 8601）	每次 metadata 变更时更新	2026-07-04T17:05:42Z
status	当前状态	用户/AI 管理	reading

type 与 kind 决定了 metadata.json 中有哪些字段，以及哪些字段是必填的。 上方是 `type: paper, kind: research` 的完整字段。下面展示如果 `type` 不同，`metadata.json` 的结构会发生什么变化。

如果是 task（任务）：

```
{
  "id": "Task_Fabricate_Flexible_Electrode",
  "type": "task",
  "kind": "todo",
  "schemaVersion": "1.0",

  "title": "制作柔性神经电极",
  "assignee": "Zhang San",
  // startDate: 任务计划开始时间，ISO 8601 格式
  // YYYY-MM-DDTHH:MM:SS → 年-月-日 T 时:分:秒
  // 2026-07-01T09:00:00 = 2026年7月1日 上午9:00
  "startDate": "2026-07-01T09:00:00",
  // endDate: 任务计划结束时间，ISO 8601 格式
  // YYYY-MM-DDTHH:MM:SS → 年-月-日 T 时:分:秒
  // 2026-08-15T18:00:00 = 2026年8月15日 下午6:00
  "endDate": "2026-08-15T18:00:00",
  "priority": 3,
  "status": "in_progress",
  "progress": 65,
  "tags": ["电极制备", "LIG"],

  "physicalPath": "研究生/任务管理/脑类器官神经电极设计与制备/"
```



```
"knowledgeParents": ["Space_脑类器官神经电极设计与制备"],

"capabilities": ["Editable", "AIUnderstandable", "Runnable"],
"createdAt": "2026-07-01T10:00:00Z",
"updatedAt": "2026-07-05T15:30:00Z"
}
```

注意与 paper 的差异：没有 **authors**、**doi**、**journal**、**volume** 等学术字段，而是 **assignee**（责任人）、**startDate/endDate**（任务计划开始/结束时间，格式为 ISO 8601 YYYY-MM-DDTHH:MM:SS，精确到分钟以满足任务规划需求）、**priority**（优先级 1-5）、**status**（任务状态）、**progress**（完成百分比 0-100）。**capabilities** 多了 **Runnable**（可执行），少了 **Exportable** 和 **Versioned**。

如果是 **space**（知识空间）：

```
{
  "id": "Space_脑类器官神经电极",
  "type": "space",
  "kind": "domain",
  "schemaVersion": "1.0",

  "behavior": ["Navigation", "KnowledgeAggregation"],

  "title": "脑类器官神经电极",
  "description": "用于脑类器官电生理记录的电极技术集合（以 1.5 节文件树中的 Space '脑类器官神经电极' 为例）",
  "tags": ["神经电极", "脑类器官"],

  "physicalPath": "研究生/文献库/脑类器官/脑类器官神经电极/",
  "knowledgeParents": ["Space_脑类器官"],

  "capabilities": ["Editable", "AIUnderstandable", "Embeddable"],
  "createdAt": "2026-06-01T08:00:00Z",
  "updatedAt": "2026-07-05T10:00:00Z"
}
```

space 的 metadata 没有 paper 的 **authors**、**doi** 等字段，也没有 task 的 **assignee**、**startDate/endDate**（精确到分钟的任务时间）、**priority**、**progress**。取而代之的是 **description**（领域描述）和 **behavior**（空间行为声明）。以上方 Space 脑类器官神经电极 为例，它的 **knowledgeParents** 指向 Space_脑类器官，表示这是一个三级子文件夹（归属链：Space_脑类器官神经电极 → Space_脑类器官 → Space_文献库 → Project_研究生）。

核心规则： **type** 决定一组**核心必填字段**和**可选字段**（数据模型骨架），**kind** 在骨架内微调。不同 type 的 metadata.json 可能只有 30%~50% 的字段重叠（如 **id**、**type**、**kind**、**title**、**tags**、**physicalPath**、**knowledgeParents**、**capabilities**、**createdAt**、**updatedAt** 是所有 type 共有的），其余字段因 type 而异。

所有 Neuron 都拥有 knowledgeParents：Space 也不例外

knowledgeParents 不是叶子节点（Paper/Task）的专属字段。每个 Neuron——无论 Space 还是叶子——都在自己的 metadata.json 中声明 knowledgeParents（父 Space 的 ID 列表）。Neuron Tree 渲染器从项目中所有 Neuron 的 knowledgeParents 取并集，通过 DFS 实时构建完整树。

以 1.5 节中的文件树为例——文献1（paper）位于 研究生/文献库/脑类器官/脑类器官神经电极/ 路径下：

Paper 的 metadata.json（叶子节点，以 文献1 为例）：

```
{
  "id": "Paper_文献1_StretchableMesh",
  "type": "paper",
  "kind": "research",
  "schemaVersion": "1.0",
  "physicalPath": "研究生/文献库/脑类器官/脑类器官神经电极/",
  "knowledgeParents": [
    "Space_脑类器官神经电极"
  ]
}
```

```
],  
  "title": "Stretchable Mesh Nanoelectronics for 3D Single-Cell Chronic Electrophysiology"  
}
```

Space 的 metadata.json（中间节点，以 脑类器官神经电极 为例）：

```
{  
  "id": "Space_脑类器官神经电极",  
  "type": "space",  
  "kind": "domain",  
  "schemaVersion": "1.0",  
  "behavior": [  
    "Navigation",  
    "KnowledgeAggregation"  
  ],  
  "physicalPath": "研究生/文献库/脑类器官/脑类器官神经电极/",  
  "knowledgeParents": [  
    "Space_脑类器官"  
  ],  
  "title": "脑类器官神经电极"  
}
```

Space 的 knowledgeParents 表达的是什么？ 它表达的是这个 Space 本身在 Neuron Tree 中的父 Space。以 1.5 节文件树为例：

- 文献1（paper）的 knowledgeParents 指向 Space_脑类器官神经电极——表示这篇论文位于"脑类器官神经电极"Space 下
- 脑类器官神经电极（space）的 knowledgeParents 指向 Space_脑类器官——表示这个 Space 本身又位于"脑类器官"Space 下
- 脑类器官（space）的 knowledgeParents 指向 Space_文献库——继续向上归属
- 文献库（space）的 knowledgeParents 指向 Project_研究生——最终归属到项目根

渲染器通过 DFS 遍历所有 Neuron 的 knowledgeParents，实时构建完整的树形结构。Space 不需要单独的 level 字段——它在树中的深度（一级/二级/三级子文件夹）由 knowledgeParents 链的长度自然决定。

关键设计：

- physicalPath：项目相对路径（只有一个），反映磁盘真实目录结构。如 研究生/文献库/脑类器官/脑类器官神经电极/，运行时拼接 Project Root
- knowledgeParents：所有 Neuron（Space + 叶子）都有的字段。存的是父 Space 的 ID 列表，而非完整路径。取全体 Neuron 的 knowledgeParents + Space 的 knowledgeParents 取并集，通过 DFS 实时构建完整的 Neuron Tree
- 物理/逻辑分离由 physicalPath 和 knowledgeParents 共同实现
- knowledgeParents 与 graph.db 的分工：knowledgeParents 是用户主动管理的导航关系（拖拽归类、手动编辑）。graph.db 只保存知识关联（Entity Relation），不保存归属导航。两者彻底分离。

为什么用 knowledgeParents 而非 knowledgePaths？

先看如果存完整路径会有什么问题。以 1.5 节文件树为例：

假设 文献1 的 knowledgePaths 是：

```
{ "knowledgePaths": ["/研究生/文献库/脑类器官/脑类器官神经电极/文献1"] }
```

这个路径中的 脑类器官神经电极 是一个 Space 的名称。当用户想把这个 Space 改名为 脑类器官柔性电极（更具体的名字）时——

用 knowledgePaths（存完整路径）：

改一个 Space 名 → 该 Space 下的**每一篇论文、每一个子任务**的 `knowledgePaths` 中，路径片段 `/脑类器官神经电极/` 都失效了。系统必须遍历该 Space 的所有子孙 Neuron，逐一更新它们的 `knowledgePaths`。若该 Space 下有 5000 篇论文，就要读写 5000 个 `metadata.json`。这是典型的**反规范化（denormalization）**——路径信息被复制到每一个 Neuron 中。

用 `knowledgeParents`（存父 Space ID）：

```

{ "knowledgeParents": [ "Space_脑类器官神经电极" ] }
```

这里的 `Space_脑类器官神经电极` 是一个 Space 的 ID，不是它的名字。Space 改名后其 ID 不变（用户修改的是 `title` 显示名，不是 `id`）。因此改名只需要修改一个文件——那个 Space 自己的 `metadata.json` 中的 `title` 字段。其下的所有论文和任务无需任何修改。Neuron Tree 渲染时通过 DFS 从 Space 出发向下遍历，Space 的 `title` 变了，树中显示的路径名就会实时更新。

类比：就像数据库中用外键（foreign key）而不是冗余的字符串来关联表。`knowledgeParents` 就是外键——它指向父 Space 的 ID，而不是把父 Space 的名字复制到每个孩子身上。

改名具体流程（以 1.5 节文件树为例）：

- 1. 用户右键 Space "脑类器官神经电极" → 选择"重命名"
- 2. 用户输入新名 "脑类器官柔性电极"
- 3. 系统只修改该 Space 的 `metadata.json`：{ `"title": "脑类器官柔性电极"` }（其 `id` 仍为 `Space_脑类器官神经电极`）
- 4. 所有子 Neuron（文献1、文献2等）的 `knowledgeParents` 依然指向 `Space_脑类器官神经电极`，无需触碰
- 5. 下一次 Neuron Tree 渲染时，DFS 读到 Space 的新 title，树中各处自动显示为 "脑类器官柔性电极"

这就是 "改名只需改一个 Space 的 `metadata`" 的含义。

谁修改 `metadata`：

操作者	Connector 写入（自动）	用户修改（可选）	说明
Connector 导入	<code>type</code> 、 <code>kind</code> 、 <code>title</code> 、 <code>authors</code> 、 <code>doi</code> 、 <code>year</code> 、 <code>journal</code> 、 <code>volume</code> 、 <code>issue</code> 、 <code>pages</code> 、 <code>issn</code> 、 <code>language</code> 、 <code>publisher</code> 、 <code>url</code> 、 <code>keywords</code> 、 <code>createdAt</code>	<code>tags</code> 、 <code>physicalPath</code> 、 <code>knowledgeParents</code>	Connector 自动抓取元数据字段。 用户可在导入时选择目录 （→ <code>physicalPath</code> + <code>knowledgeParents</code> ）和添加标签 （→ <code>tags</code> ）
操作者	可修改字段	是否需要用户确认	
用户	所有字段	—（用户主动操作）	
插件	按其权限范围内字段	视插件设计而定	
AI（自动）	<code>tags</code> （自动打标签）、 <code>status</code> （状态推断）	✅ 必须通知用户确认	
AI（用户命令）	<code>physicalPath</code> 、 <code>knowledgeParents</code> （用户要求AI整理目录时）	✅ 必须报告修改并待用户确定	

Metadata 判定原则

核心原则：可作为 View 筛选条件的数据，一律属于 `metadata`。

这一原则提供了 `metadata.json` 与 `Knowledge.md`、`graph.db` 之间的明确边界：

数据示例	是否属于 metadata	原因
负责人（assignee）	✓	Table/Kanban View 可按负责人筛选、分组
开始时间（startDate）	✓	Timeline/Calendar View 的时间轴筛选基准
截止时间（endDate）	✓	Calendar View 范围筛选、甘特图排序
状态（status）	✓	Kanban View 的列分组依据
优先级（priority）	✓	Table View 排序键、Kanban 着色依据
标签（tags）	✓	跨 View 通用的筛选和聚合维度
任务完成进度（progress）	✓	Table View 排序、进度条渲染
type / kind	✓	View 的类型过滤器（"只看 paper""只看 task"）
论文的期刊（journal）	✓	Table View 按期刊分组、期刊投稿分析模板
论文的发表年份（year）	✓	Timeline View 的时间轴、按年份浏览模板

跨层筛选的处理：

如果筛选需求涉及关系层数据（如"筛选所有属于 Harvard 课题组的论文"），View 引擎做 **join 查询**——同时查询 `metadata.json`（获取论文基础字段）和 `graph.db`（获取 `BelongTo` 关系），结果合并后渲染。关系数据**不冗余写入 metadata**，保持四层架构的职责清晰。

例外——knowledgeParents：虽然"按 Space 筛选"是最常用的 View 筛选项，但 `knowledgeParents` 已经存在于 `metadata.json` 中（因为它是 `Neuron` 身份定义的一部分），不需要额外处理。

physicalPath 的设计规则

`physicalPath` 存的是 **Project Relative Path（项目相对路径）**，而非绝对路径或带盘符的完整路径。

```
{
  "physicalPath": "研究生/文献库/脑类器官/脑类器官神经电极/"
}
```

运行时由 `Project Root + relativePath` 拼接得到真正的磁盘路径。好处：

- **跨平台兼容：**Windows (`D:\Research\...`)、Mac (`/Users/name/...`)、Linux 均只需替换 Project Root 即可
- **Git 友好：**相对路径在仓库内一致，不会因不同机器的绝对路径产生 diff
- **项目可迁移：**整体移动项目目录时，所有 `physicalPath` 无需修改

Manifest（能力清单）

`metadata` 中增加 `capabilities` 字段，声明该 `Neuron` 支持的行为能力：

```
{
  "id": "Task_Fabricate_Flexible_Electrode",
  "type": "task",
  "kind": "todo",
  "schemaVersion": "1.0",
  "title": "制作柔性神经电极",
  "assignee": "Zhang San",
  // startDate: YYYY-MM-DDTHH:MM:SS → 年-月-日 T 时:分:秒
  "startDate": "2026-07-01T09:00:00",
  // endDate: YYYY-MM-DDTHH:MM:SS → 年-月-日 T 时:分:秒
  "endDate": "2026-08-15T18:00:00",
  "priority": 3,
  "status": "in_progress",
  "progress": 65,
```

```
"capabilities": [
  "Editable",
  "Embeddable",
  "AIUnderstandable",
  "Runnable"
]
```

常见能力值：

Capability	含义	适用类型
Editable	用户可编辑内容	所有 Neuron
Embeddable	可嵌入到其他 View（如 Timeline）	task, paper
AIUnderstandable	AI 可读取 Knowledge.md 理解	所有 Neuron
Runnable	可执行（如 Workflow）	task, protocol
Playable	可播放（视频/音频）	video
Exportable	可导出为标准格式	paper, dataset
Versioned	支持版本管理	paper, protocol

原则：AI 对 metadata.json 的任何修改都必须通知用户，待用户确认后生效。 这是防止 AI 静默改变用户数据结构的关键约束。

1.4 Knowledge.md —— 知识层（Knowledge Layer）

职责： 回答"它意味着什么（What it means）"。

这是 Neuro 真正的创新所在。它不是笔记，不是摘要，而是 **AI 持续维护的活知识索引（Living Knowledge）**。

设计原则：Knowledge.md 只写知识——不写 metadata。

为什么？ Title、Tags、Status、Last Updated 是 metadata.json 的职责。Knowledge.md 专注于那些 **不能被结构化字段表达的理解**——为什么做、怎么做、总结、思考、经验。metadata 归 metadata，知识归 Knowledge.md。

Knowledge.md 永远只包含以下五个章节：

```
# 总结（Summary）    ← 一句话：这是关于什么的
# 为什么（Why）      ← 动机、背景、问题
# 怎么做（How）      ← 方法、步骤、关键参数
# 思考（Thoughts）   ← AI 洞察、分析、推理
# 经验（Experience） ← 教训、可复用知识、关联启发
```

不同 type 的 Neuron 在五个章节中填充的内容有所不同，但结构完全统一：

Paper 类型示例：

```
# 总结
本文首次提出用 LIG 方法制备柔性神经电极用于脑类器官记录，实现 3.2kΩ 低阻抗和长期稳定记录。

# 为什么
脑类器官的电生理记录需要柔性、低阻抗、可长期植入的电极。传统金属微丝阵列创伤大、无法贴合曲面。LIG（激光诱导石墨烯）方法提供了材料可设计性和工艺简便性的新可能。

# 怎么做
- 电极材料：LIG（激光诱导石墨烯）
```

- 阻抗：3.2kΩ @ 1kHz
- 电极直径：50μm
- 柔性基底：PI（聚酰亚胺）
- 制备流程：PI 清洗 → 激光参数优化 → LIG 图案化 → 绝缘封装 → 阻抗测试 → 细胞培养验证

思考

LIG 的阻抗优势来自于其多孔结构增大了有效表面积。但多孔结构也带来机械强度问题——在长期培养中可能出现微裂纹。后续研究应关注 LIG 与基底的界面结合力优化。
该论文的铂黑电镀方案与 [[任务1: 学习电镀铂黑]] 直接相关——用户可在完成该任务后对比自己的电镀结果与论文中的阻抗值。

经验

- 激光功率与扫描速度的组合对 LIG 质量影响极大，建议先做功率矩阵实验（参考 [[Experiment_001]]）
- PI 基底清洗不彻底会导致 LIG 附着力差，超声清洗 10min + 等离子处理 5min 效果最佳
- 阻抗测试必须在 PBS 缓冲液中进行（而非干燥状态），否则结果不可比
- 细胞培养验证周期至少 2 周才能评估长期生物相容性

Task 类型示例：

总结

完成 LIG 神经电极的完整制备与测试流程，为脑类器官电生理记录提供可靠的电极方案。

为什么

脑类器官项目需要可定制的柔性神经电极。现有商业电极（MEA）间距过大（200μm），无法实现单细胞分辨率。自制 LIG 电极可将间距缩小至 50μm，且成本仅为商业方案的 1/10。

怎么做

- [x] PI 基底清洗 ← 已完成 → metadata.progress += 15%
- [x] 激光参数初步优化 ← 已完成 → metadata.progress += 20%
- [] LIG 制备与转移 ← 进行中
- [] 阻抗测试 ← 待开始
- [] 细胞实验验证 ← 待开始

详细参数参考 [[Paper_A]] 与 [[Experiment_001]]。

思考

当前瓶颈在激光参数优化——功率过高导致 PI 基底碳化过度，功率过低则 LIG 导电性不足。建议先完成功率矩阵实验（5W~15W，步长 1W），找到最佳窗口。

经验

- CNT（碳纳米管）分散性不足是常见问题——超声时间延长至 30min 可显著改善
- 激光路径的填充间距（hatch spacing）比功率更影响均匀性——先优化间距再调功率
- 阻抗测试前务必校准电极，否则数据无法与文献值对比

Knowledge.md 中的 Todo List 与 metadata.json 中的 progress 字段实时双向同步：

- 当用户（或 AI）在 Knowledge.md # 怎么做 中勾选 [] → [x] 时，系统自动重新计算 progress = 已完成项数 ÷ 总项数 × 100，回写到 metadata.json
- 当 AI 更新 metadata.json 的 progress 时（如用户告知“阻抗测试做了一半”），系统将此百分比映射到 Todo List 的勾选状态
- 这一同步机制确保了甘特图、Kanban 等视图始终反映最新进度

所有 Knowledge.md 必须包含的统一结构

每个 Knowledge.md 无论 type 是什么，都必须包含以下五个章节：

- # 总结（Summary） ← 一句话总结（必填）
- # 为什么（Why） ← 动机与背景（必填）
- # 怎么做（How） ← 方法、步骤、参数（必填）
- # 思考（Thoughts） ← AI 洞察、分析（必填）
- # 经验（Experience） ← 教训、可复用知识、关联启发（必填）

各 type 在五个章节中的填充内容：

Type	总结 (Summary)	为什么 (Why)	怎么做 (How)	思考 (Thoughts)	经验 (Experience)
paper	研究结论	研究动机/问题	方法/关键参数	AI 分析/洞察	教训/关联文献
task	任务目标	背景/动机	Todo List/步骤	AI 建议/思路	风险/教训/依赖
experiment	实验结果	实验目的	材料/方法/参数	分析/推断	教训/可复用流程
meeting	会议结论	会议背景	议程/要点	洞察/分析	待办/后续行动
idea	核心想法	灵感来源	实现路径	可行性分析	相关参考
note	核心内容	记录背景	详细内容	思考/分析	关联/启发
space	领域概览	领域意义	(空或研究路径)	趋势/热点	推荐阅读/资源
person	研究方向	学术背景	方法/技术栈	合作潜力	相关项目/成果
group	课题组概况	成立背景	研究方向	领域影响力	代表性成果
webpage	页面核心观点	浏览背景	内容摘要	分析/评价	相关链接/资源

AI 查询策略：

- AI 通过 `metadata.json` 中的 `type` 字段判断需要查询哪种 Neuron
- 对于跨类型搜索（如"找出所有与 LIG 相关的内容"），AI 优先读取 `# 总结`（快速筛选）→ 命中后深入读取 `# 怎么做` 和 `# 思考`（获取细节）
- `# 经验` 中的 `[[...]]` 链接和 `[text](Assets/...)` 附件引用由 Markdown 解析器自动检测并写入 graph.db 的 LinksTo 关系

关键特性：

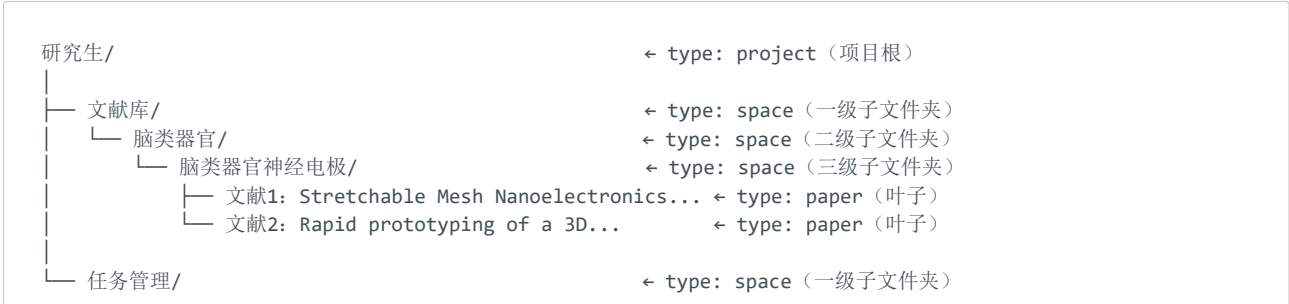
- **metadata 归 metadata**：Title / Tags / Status / Last Updated 全部在 metadata.json 中。Knowledge.md 不重复这些字段，由程序在渲染时从 metadata 读取并展示（如顶部显示标题和状态）。
- **持续成长**：AI 每天可以更新五个章节中的内容，用户也可以补充
- **AI 优先读取**：所有 AI 交互首先读取 Knowledge.md 的五个章节，无需打开 PDF
- **Token 优化**：Knowledge.md 本身就是经过提炼的知识精华——不含 metadata 噪音
- **人机协同**：用户和 AI 共同编辑五个章节

1.5 Knowledge Space（知识空间）

定义： `type: "space"` 的 Neuron。

Knowledge Space 是一种特殊的 Neuron，它的职责是**组织和聚合**其下的其他 Neuron。

下面用一个具体的文件树示例来说明 Space 与叶子 Neuron 的关系：



└─ 脑类器官神经电极设计与制备/
└─ 任务1: 学习电镀铂黑
└─ 任务2: 制备柔性神经电极

← type: space (二级子文件夹)
← type: task (叶子)
← type: task (叶子)

在这个文件树中：

- **Space (type: space)** 是中间节点，负责组织和聚合：**文献库**、**脑类器官**、**脑类器官神经电极**、**任务管理**、**脑类器官神经电极设计与制备**——它们都是 Space
- **叶子 Neuron** 是终端知识对象：**文献1** (paper)、**文献2** (paper)、**任务1** (task)、**任务2** (task)
- **Space 可以嵌套**：**脑类器官神经电极** 是 **脑类器官** 的子 Space，**脑类器官** 又是 **文献库** 的子 Space，形成了三级嵌套

关于 Space 层级的设计决定： Space 的层级不通过 **type** 字段区分"一级""二级""三级"，而是由 **knowledgeParents** 组成的树状结构隐式表达。渲染器通过 DFS 遍历自动计算层级。**不需要额外的 level 字段来描述当前 Space 所属的文件夹等级**——一级、二级、三级子文件夹的角色由它在 knowledgeParents 树中的深度自然决定：

- **文献库** 的 knowledgeParents 指向 **Project_研究生** (深度 1 → 一级子文件夹)
- **脑类器官** 的 knowledgeParents 指向 **Space_文献库** (深度 2 → 二级子文件夹)
- **脑类器官神经电极** 的 knowledgeParents 指向 **Space_脑类器官** (深度 3 → 三级子文件夹)

如果引入 **level** 字段，每次拖拽 Space 到树的不同深度时都需要手动更新该值，反而引入不一致风险。保留 knowledgeParents 作为唯一的归属描述源，层级由渲染器实时计算，是最简洁可靠的设计。

关于课题组： 课题组是 **type: group** 的 leaf Neuron，位于 **Webpage/ Space** 下。合作网络、论文归属通过 **.neuro/graph.db** 中的 **BelongTo** 和 **CollaboratesWith** 关系表达，而非目录结构。

Space 的 Knowledge.md 同样遵循五章结构——但它综述的是**整个领域**，而非单篇论文：

总结

共有论文 132 篇，实验 21 项，任务 12 个。该领域正处于从刚性电极向柔性、可拉伸电极过渡的阶段。

为什么

脑类器官作为体外 3D 神经模型，需要能够长期、无损地记录其电活动。传统 MEA 和金属微丝阵列在柔性和空间分辨率上存在根本限制。

怎么做

(本 Space 下所有 Paper 和 Experiment 的方法聚合——由 AI 自动归类整理)

思考

当前研究热点集中于三个方向：材料创新 (LIG/PEDOT:PSS/CNT)、结构创新 (网状/多孔/3D 打印)、集成创新 (无线传输/片上信号处理)。LIG 路线因其工艺简便性和材料可设计性正快速崛起。

经验

- 推荐阅读：[[文献1: Stretchable Mesh]] (柔性网状电极的开创性工作)、[[文献2: Rapid prototyping]] (3D 打印多孔电极的低成本路线)
- 关键对比维度：阻抗、机械柔性、长期稳定性、制备复杂度、成本

Space 使 Neuro 能够管理"知识的空间"本身，这是与 Obsidian、Notion、Zotero 最大的区别。

Space 的 Behavior (行为声明)

Space 在 metadata 中通过 **behavior** 字段声明自己的行为角色，解决 Space 承担"导航目录"和"领域知识"双重角色的问题：

```
{  
  "id": "Space_2026",  
  "type": "space",  
  "kind": "category",  
  "behavior": ["Navigation"],  
}
```



```
"title": "2026"
}
```

Behavior	含义	AI 行为	示例
Navigation	纯导航目录	不自动生成综述/图谱	2026/、Inbox/、Archive/
KnowledgeAggregation	研究领域集合	AI 自动维护该 Space 的 Knowledge.md（综述、趋势）	Brain Organoid/、LIG/
Both	两者兼具	导航 + AI 主动分析	大多数研究型 Space

原则： 仅 KnowledgeAggregation 或 Both 的 Space 才会触发 AI 知识整理，避免为纯目录生成无价值的综述。

Space 的 Cover 封面图

Space 可拥有 Cover.png 作为视觉标识，存储在 Assets/Cover.png：

```
Space_BrainOrganoid/
├─ metadata.json
├─ Knowledge.md
└─ Assets/
    └─ Cover.png          ← AI 生成 / 用户上传的封面图
```

Cover 用于 Gallery View 卡片墙、Space 详情页头部展示等场景。

Space Knowledge.md 的更新机制

设计原则： Space 的 Knowledge.md 不会频繁自动更新，仅在以下情况触发：

- 1. **创建时：** 提供 Space 模板，允许用户手动编写初始内容
- 2. **子树变更时：** 当 Space 下的 Neuron 数量/类型发生变化时，程序检测到变化，提示用户是否更新 Knowledge.md
- 3. **手动触发：** 用户可随时右键 Space 选择"更新 Knowledge.md"
- 4. **AI 辅助：** 用户可选择"AI 自动更新"或"手动编写"

Token 策略： AI 综述的 Token 消耗由用户承担（计入用户的 AI API 费用），因此不做无意义的自动轮询更新。

1.6 Relation（关联关系）与 .neuro/ Runtime

Relation 是 Neuron 之间的连接，构成知识图谱。

重要升级： Relation 不存储在 Neuron 目录内的 JSON 文件中，而是存储在项目级的 .neuro/ 运行时数据库中。

Graph 不是图片。Graph 就是数据库。

1.6.1 .neuro/ 是什么

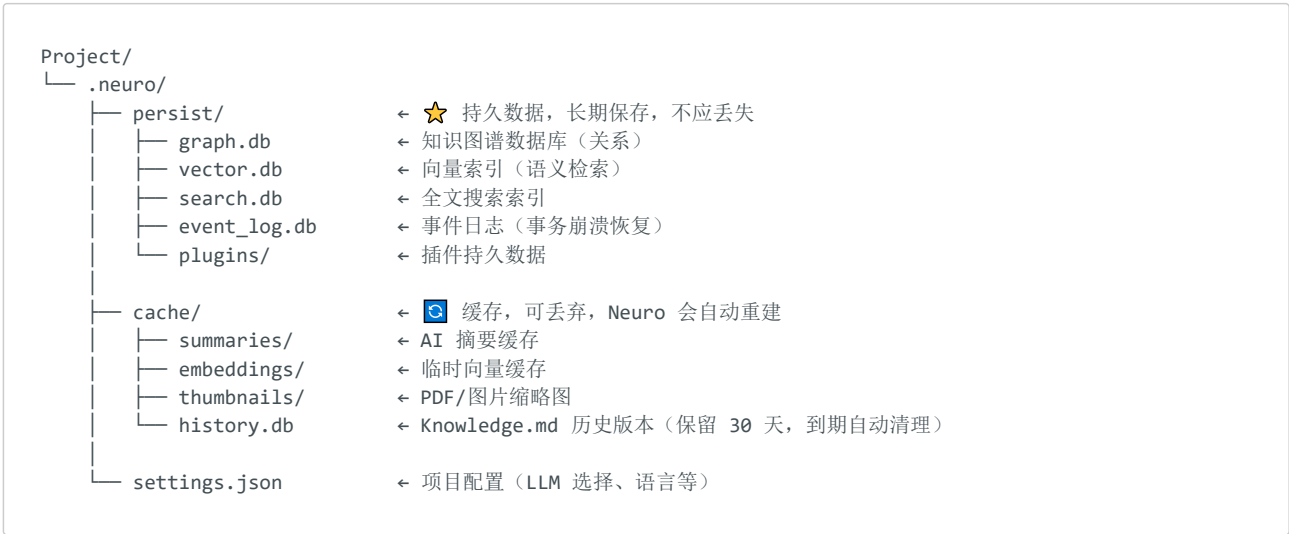
.neuro/ 不是用来存"笔记内容"的，它存的是 Neuro 系统的"运行时状态"——关系数据库、向量索引、全文搜索索引、AI 缓存等。你的知识内容（论文笔记、任务信息、课题组描述等）存储在 Neuron 目录的 Knowledge.md 和 metadata.json 中。

类比 .git/，.neuro/ 是 Neuro 的运行时目录，用户默认不可见。

.neuro/ 不能进入版本控制 (.gitignore)，因为它包含大量二进制数据和频繁变更的缓存，会导致 Git 冲突和无意义的 diff。

1.6.2 .neuro/ 内部结构：持久数据 vs 缓存

.neuro/ 内部按生命周期划分为两层：



search.db 里存了什么？

search.db 的内部结构（概念示意）：

FTS 索引表：对项目中所有 Neuron 的以下文本建立倒排索引：

- metadata.json 中的 title、tags、description 等字段

- Knowledge.md 全文（# 总结、# 怎么做、# 思考 等）

倒排索引原理示意：

"阻抗" → [文献1_metadata, 文献1_Knowledge.md, 文献2_Knowledge.md, 任务2_Knowledge.md]

"LIG" → [文献1_metadata, 文献1_Knowledge.md]

"电镀铂黑" → [任务1_Knowledge.md]

查询 search.db → "阻抗" → 返回结果：

Neuron ID	匹配文件	匹配位置
Paper_文献1_StretchableMesh	Knowledge.md	# 怎么做 → 阻抗: 3.2kΩ
Paper_文献2_RapidPrototyping	Knowledge.md	# 怎么做 → 阻抗: 5.1kΩ
Paper_文献2_RapidPrototyping	metadata.json	tags: ["阻抗测试", ...]

Neuro 拿到的结果是：文献1 和 文献2 涉及阻抗。

第三步：为什么还需要 vector.db？——语义搜索

查询来源：.neuro/persist/vector.db（向量语义索引）

search.db 只能做精确文本匹配——搜"阻抗"只能找到写了"阻抗"二字的文件。但如果一篇论文写的是"electrode impedance"（英文）或"电极电阻"，search.db 就匹配不到。

vector.db 解决的就是这个问题。它是一个向量嵌入数据库，存储了每个 Neuron 的 Knowledge.md 内容经过 Embedding 模型（如 text-embedding-3-small）转化后的向量。

vector.db 里存了什么？

vector.db 的内部结构（概念示意）：

对每个 Neuron 的 Knowledge.md（或 metadata 关键字段）
计算一个 1536 维的浮点数向量（embedding）：

文献1 → [0.023, -0.145, 0.891, ..., 0.034] (1536维)

文献2 → [0.018, -0.132, 0.876, ..., 0.041]

任务1 → [0.756, 0.321, -0.034, ..., 0.112]

语义相近的内容 → 向量距离近

语义无关的内容 → 向量距离远

当用户搜"阻抗"时，Neuro 可以同时做：

- 精确搜索 (search.db)：返回所有包含"阻抗"文字的文件
- 语义搜索 (vector.db)：将"阻抗"转为向量，在 vector.db 中找最相似的前 K 个 Neuron → 可能返回写有 "electrode impedance" 但没有写"阻抗"二字的论文

两步结果合并，确保不会漏掉相关内容。

第四步：提取具体数值——读原始文件

Neuro 已确定 文献1 和 文献2 涉及阻抗。现在需要取出实际的阻抗数值来排序。

这一步不经过 .neuro/——直接读 Knowledge.md：

文献1 Knowledge.md → # 怎么做 → - 阻抗：3.2kΩ
文献2 Knowledge.md → # 怎么做 → - 阻抗：5.1kΩ

AI 解析出数值 → 排序 → 结果：文献1 (3.2kΩ) < 文献2 (5.1kΩ)。

第五步：知识关系——跨论文的关联

查询来源：.neuro/persist/graph.db（关系数据库）

graph.db 存储的是 Neuron 之间的知识关系（注意：不是导航关系，导航由 knowledgeParents 处理）：

graph.db 的核心原则：只存 Entity Relation，不存 Navigation。

relations 表：		
source_id	target_id	type
Paper_文献1	Paper_文献2	Cite
Experiment_阻抗验证	Paper_文献1	Support
Paper_文献1	Harvard_Liu_Lab	BelongTo
Task_制备柔性神经电极	Paper_文献1	DependsOn

在本次阻抗排行榜查询中，graph.db 可以提供额外信息：比如发现 文献1 和 文献2 之间存在 Cite 关系，在排行榜中显示"文献1 引用了 文献2"。

为什么 knowledgeParents 和 graph.db 要分离？ 见下方对照表：

维度	knowledgeParents（metadata 字段）	graph.db（.neuro/ 运行时）
存什么	父 Space ID 列表	知识关系（Cite/Support/BelongTo/DependsOn 等）
谁管理	用户（拖拽归类、手动编辑）	AI（自动发现）+ 用户确认
驱什么	Neuron Tree 渲染（导航）	知识图谱（Graph View）、课题组地图
例子	文献1 → knowledgeParents: [Space_脑类器官神经电极]	文献1 → Cite → 文献2

两者彻底分离：导航由用户管理（拖拽归类），知识关联由 AI 管理（自动发现）。

第六步：缓存——加速下一次查询

查询来源：.neuro/cache/

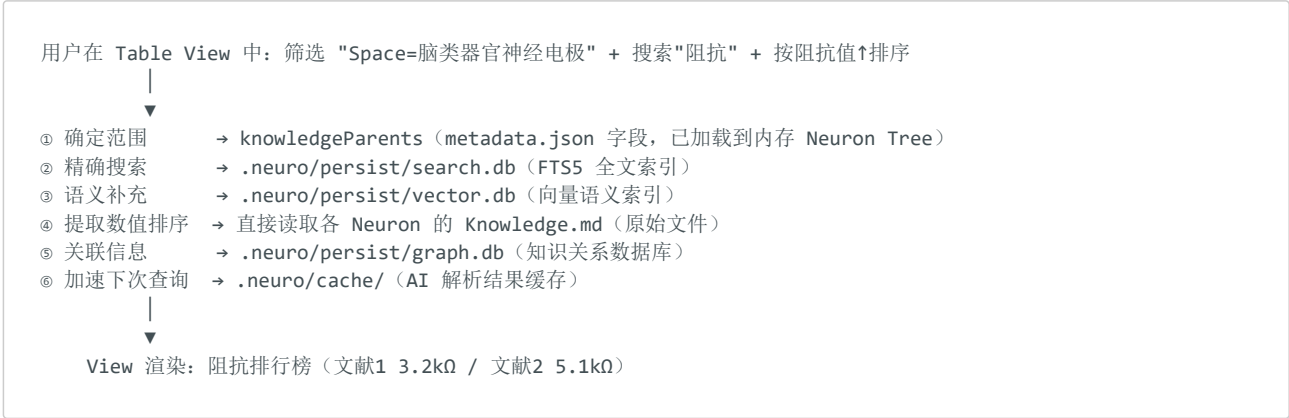
AI 解析 Knowledge.md 中的阻抗数值需要消耗 Token。Neuro 把解析结果缓存下来：

缓存位置	缓存内容
summaries/	"文献1 的关键参数：阻抗 3.2kΩ, 电极直径 50μm" — AI 已解析过的结构化摘要
embeddings/	文献1 的临时向量（当 vector.db 尚未重建时使用）

缓存位置	缓存内容
thumbnails/	文献1 的 PDF 首页缩略图，Gallery View 展示用
history.db	Knowledge.md 的修改历史（30天滚动），支持"恢复到 3 天前的版本"

下次查询"阻抗排行榜"时，如果 summaries/ 中已有解析结果，直接读取缓存，**不再消耗 Token**。

完整流程总结：



关键理解：.neuro/ 保存的是"索引"和"关系"，而不是"内容本身"。

- 内容（论文摘要、任务目标、实验数据）→ 存在 Neuron 目录的 Knowledge.md 和 metadata.json 中
- 索引（"阻抗"这个词在哪些文件里出现）→ 存在 .neuro/search.db 中
- 语义映射（哪些文件讨论的是同一类话题）→ 存在 .neuro/vector.db 中
- 关系（A 引用了 B，C 属于 D 课题组）→ 存在 .neuro/graph.db 中
- 缓存（AI 已解析过的结构化结果，下次直接用）→ 存在 .neuro/cache/ 中

.neuro/ 让 Neuro 能做到"搜索 10000 篇论文并排序"在秒级完成，而不是逐篇 grep。

1.6.4 为什么用数据库而非文件来存关系和索引？

对比维度	文件方案（JSON/Markdown）	数据库方案（.neuro/ SQLite）
查询"某 Neuron 的所有邻居"	遍历所有 Neuron 文件逐一检查	<code>SELECT * FROM relations WHERE source_id = ?</code> — 0.001 秒
全文搜索"阻抗"	逐文件 grep，10000 篇论文耗时长	FTS5 倒排索引，毫秒级
语义搜索"类似话题"	需要实时读取并计算所有文件的 embedding	vector.db 预计算向量 + 近似最近邻搜索
关系量级	10 万条关系散落为 10 万个文件时无法管理	SQLite 单表轻松承载百万行
跨关系查询	几乎不可能（"找出 A 引用且 B 支持的论文"）	SQL JOIN 一行语句

1.6.5 索引的创建与更新时机

search.db 和 vector.db 不是在用户第一次查询时才创建的——它们在 Neuron 的**创建阶段（Create）和理解阶段（Understand）**就已经建立。具体时机如下：

① Connector（采集）
浏览器插件/Zotero 同步/手动上传

Neuron 目录 + metadata.json + Assets/ 归档完毕

↓

🟢 search.db 初次索引（同步，毫秒级）
提取 metadata.json 的 title/tags/description/abstract 等文本字段
写入 FTS5 倒排索引

↓

🟡 Knowledge.md 生成完毕（AI 理解阶段）
此时 Knowledge.md 已包含 # 总结、# 思考 等完整内容

↓

🟢 search.db 增量更新（将 Knowledge.md 全文加入索引）
🟠 vector.db 生成向量（异步，AI API 调用）
对 Knowledge.md 全文 + metadata 关键字段计算 Embedding
写入向量数据库，关联 Neuron ID

↓

🔄 后续每次 Knowledge.md 更新时：

- search.db: 增量更新修改过的文本（FTS5 支持增量写入，毫秒级）
- vector.db: 标记向量为"过期"，在后台队列中异步重新计算新向量（不会阻塞用户操作，用户可能在几秒内看到语义搜索结果略有滞后）

触发时机	search.db（全文索引）	vector.db（语义向量）
Neuron 创建（metadata.json 写入）	✅ 立即索引 metadata 文本字段	❌ 等待 Knowledge.md 生成
AI 生成 Knowledge.md 初版	✅ 立即索引 Knowledge.md 全文	✅ 异步计算 Embedding
Knowledge.md 被编辑（用户/AI）	✅ 增量更新（毫秒级）	⚠️ 标记过期 → 后台队列重建
metadata.json tags/title 变更	✅ 增量更新	⚠️ 标记过期
cache/ 被清空	❌ 不影响（索引独立于缓存）	❌ 不影响
Neuron 被删除	✅ 从索引中移除	✅ 从向量库中移除

关键设计：search.db 的更新是**同步且低成本**的（FTS5 增量索引在毫秒级完成），用户编辑 Knowledge.md 后立即可以全文搜索到新内容。vector.db 的 Embedding 计算依赖外部 AI API（有耗时和费用），因此走**异步后台队列**——用户编辑后，语义搜索结果可能在 10~60 秒内略有滞后，但不会阻塞界面。用户可以在 Status Bar 看到 "向量索引重建中 (12/156)" 的进度提示。

1.7 View（视图）

View 是 Neuron 的可视化呈现方式。统一称为 View，不再使用"模式"（Mode）的叫法。

所有外部观察形式都是 View——Markdown 渲染、图谱、看板、日历、时间线、表格等。

1.8 物理组织与逻辑组织分离

这是 Neuro 数据模型中最重要的设计原则之一。

物理组织（Physical Organization）：

- 硬盘上的实际目录结构
- 一个 Neuron 只能有一个物理位置
- 由 `physicalPath` 记录

逻辑组织（Logical Organization）：

- Neuro 内部的 Neuron Tree 展示
- 一个 Neuron 可以出现在多个逻辑路径中

- 由 `knowledgeParents` 记录（父 Space 的 ID 列表，渲染器 DFS 构建完整树）

示例（以 1.5 节文件树中的 文献1 为例）： 一篇关于"脑类器官拉伸网状纳米电子学"的论文

物理存储（硬盘上的真实位置，只有一个）：

研究生/文献库/脑类器官/脑类器官神经电极/文献1/

逻辑路径（Neuron Tree 中显示， 文献1 同时出现在两个位置）：



注意： 文献1 同时出现在 脑类器官神经电极 和 脑类器官神经电极设计与制备 两个 Space 下，但硬盘上只有 研究生/文献库/脑类器官/脑类器官神经电极/文献1/ 一份数据。第二个投影是纯逻辑的—— 文献1 的 `knowledgeParents` 中增加了 `Space_脑类器官神经电极设计与制备`，不需要复制文件。

删除操作：

- 删除分类（从知识路径移除）→ 仅从 `knowledgeParents` 中移除一项，硬盘数据不变
- 删除 Neuron → 删除整个 Neuron 文件夹（`metadata.json` + `Knowledge.md` + `Assets/`）

physicalPath 与 knowledgeParents 的完整规则表

注意： `physicalPath` 为 Project Relative Path（如 `Paper/BrainOrganoid/Electrode/`），非绝对路径。运行时拼接 Project Root 得到实际磁盘路径。

操作	影响 <code>physicalPath</code>	影响 <code>knowledgeParents</code>	说明
Neuron 首次创建	✔ 写入	✔ 写入	两者初始一致
拖拽 Neuron 到新 Space	✘	✔ 追加	纯逻辑操作，不碰磁盘
右键"迁移实际存储位置"	✔ 更新	✘	显式物理搬文件
删除最后一条匹配的 <code>knowledgeParent</code>	✔ 提示迁移	✔ 删除	软件弹框："仅剩一个 <code>knowledgeParent</code> 且与 <code>physicalPath</code> 不符，是否迁移？"，用户确认后搬磁盘
重命名/移动 Space	✘	无需操作	<code>knowledgeParents</code> 存的是 ID，改 Space 名不影响其 ID，无需批量更新
AI 自动归类	✘	✔ 写入（待确认）	AI 标记为"待用户确认"，用户确认后生效

边界情况处理：

- 当用户直接操作文件系统（在文件管理器中移动/删除文件夹）导致 `physicalPath` 失效时，Neuro 自动标记为"丢失"状态，报错让用户处理，并提示用户："如不清楚后果，建议不要直接操作物理文件。"
- 所有对 `physicalPath` 的修改都应该通过 Neuro 界面完成，而非文件管理器。

1.9 内部链接与双向引用（Neuro-Link）

这是 Neuro 借鉴 Obsidian 的 `[[wikilink]]` 双向链接机制并加以扩展的核心功能。Neuro-Link 不仅实现 Neuron 之间的跳转，还支持直接跳转到 Assets/ 中的附件文件、唤起浏览器打开网页，并且链接本身也被纳入 AI 查询的索引体系。

1.9.1 链接语法

Neuro 的 Knowledge.md 支持三种链接语法：

```
# 文献1: Stretchable Mesh Nanoelectronics 的 Knowledge.md

# 总结
本文首次提出用可拉伸网状纳米电子学实现脑类器官的 3D 单细胞长期电生理记录。

# 为什么
脑类器官研究需要长期、无损的电生理记录手段，而传统 MEA 和金属微丝阵列在柔性和分辨率上存在根本限制。

# 怎么做
- 电极材料：铂黑 (Pt-Black)
- 阻抗：3.2kΩ @ 1kHz
- 电极直径：50μm
- 柔性基底：SU-8
制备方法详见： [制备流程详见 Supplement](Assets/Supplement_Methods.pdf)
原始数据表格： [阻抗测试原始数据](Assets/Impedance_Raw_Data.xlsx)

# 思考
该论文的铂黑电镀方案与 [[任务1：学习电镀铂黑]] 直接相关—
用户可在完成该任务后对比自己的电镀结果与论文中的阻抗值。

# 经验
- [[文献2: Rapid prototyping]]：同样使用铂黑，但采用 3D 打印微孔阵列
- [[任务2：制备柔性神经电极]]：本论文的 SU-8 基底方案是该任务的参考
- 论文原文： [Advanced Materials, 2022](https://onlinelibrary.wiley.com/doi/abs/10.1002/adma.202106829)
```

三种链接类型：

链接语法	含义	点击行为	示例
<code>[[Neuron_ID]]</code>	链接到另一个 Neuron	在 Workspace 中打开该 Neuron 的 Knowledge.md	<code>[[任务1：学习电镀铂黑]]</code>
<code>[[Neuron_ID#section]]</code>	链接到另一个 Neuron 的指定章节	打开并滚动到对应标题位置	<code>[[文献2#怎么做]]</code>
<code>[显示文本](Assets/文件名)</code>	链接到本地 Assets/附件	用系统默认程序打开文件（PDF → 内置阅读器，xlsx → Excel，mp4 → 视频播放器）	<code>[制备流程](Assets/Supplement_Methods.pdf)</code>
<code>[显示文本](https://URL)</code>	外部网页链接	唤起系统默认浏览器（或 Neuro 内置浏览器）	<code>[Advanced Materials, 2022](https://...)</code>

对于 `type: webpage` 的 Neuron：其 Knowledge.md 中可以使用 `[打开原网页](URL)` 链接。点击后，如果 Neuro 有内置浏览器则在内置浏览器中打开；否则唤起系统默认浏览器。这与

`metadata.json` 中的 `url` 字段呼应——`metadata.url` 记录网页来源，`Knowledge.md` 中的链接则方便用户在阅读时直接跳转。

1.9.2 反向链接（Backlinks）

Obsidian 的双向链接之所以迷人，在于不仅 A 可以链接到 B，B 也能自动感知"A 链接了我"。Neuro 同样实现了这个机制，但它不是靠扫描 Markdown 文件——而是靠 **graph.db**。

实现原理：

- ① 用户（或 AI）在 文献1 的 `Knowledge.md` 中写入：
"参考 [[任务1: 学习电镀铂黑]]"
- ② Neuro 的 Markdown 解析器在保存时自动检测 `[[...]]` 语法
→ 解析出链接目标 Neuron ID: 任务1_学习电镀铂黑
- ③ 在 `graph.db` 的 `relations` 表中写入一条关系：
source_id: Paper_文献1_StretchableMesh
target_id: Task_任务1_学习电镀铂黑
type: LinksTo
metadata: { sourceSection: "# 怎么做", context: "电极电镀参数参考" }
- ④ 用户打开 任务1 的 `Knowledge.md` 时，
Neuro 查询 `graph.db`:
`SELECT * FROM relations WHERE target_id = 'Task_任务1_学习电镀铂黑' AND type = 'LinksTo'`
→ 发现 文献1 链接了它
- ⑤ 任务1 的 `Knowledge.md` 底部自动渲染 "Backlinks" 区块：
🔄 Backlinks
- [[文献1]] #Key Findings: "电极电镀参数参考"

Backlinks 展示效果（任务1 的 `Knowledge.md` 底部）：

```
# 任务1: 学习电镀铂黑 的 Knowledge.md

...（任务内容）...

---
## 🔄 Backlinks（反向链接，自动生成）
- [[文献1: Stretchable Mesh]] → #怎么做: "电极电镀参数参考"
- [[文献2: Rapid prototyping]] → #怎么做: "铂黑电镀对比"
- [[任务2: 制备柔性神经电极]] → #怎么做: "前置任务: 电镀铂黑"
```

关键设计：

- Backlinks 区块由 Neuro 自动生成，不存储在 `Knowledge.md` 文件中——它是 `graph.db` 的实时查询结果
- 当链接被删除时（`[[...]]` 从源 `Knowledge.md` 中移除），`graph.db` 中对应的 `LinksTo` 关系同步删除，Backlinks 区块自动更新
- `LinksTo` 是 `graph.db` 中的一种新关系类型，与 `Cite/Support/DependsOn` 并列
- 同一个 Neuron 被多个 Neuron 链接时，Backlinks 按链接日期倒序排列

1.9.3 链接如何参与 AI 查询

链接本身不是 AI 查询的索引（查询索引仍是 `search.db` 和 `vector.db`），但链接信息通过 `graph.db` 增强了 AI 查询的质量：

场景一：AI 发现 `Knowledge.md` 信息有限，需要分析附件

用户提问：文献1 的电极是怎么制备的？具体步骤是什么？

AI 流程：

- ① 读取 文献1 的 Knowledge.md
 - # 怎么做 只写了"详细制备方法见支撑文件"
 - 信息不足
- ② AI 读取 Knowledge.md 中的链接:
 - [制备流程详见 Supplement](Assets/Supplement_Methods.pdf)
 - 发现指向 Assets/Supplement_Methods.pdf 的链接
- ③ AI 通过 graph.db 获取该链接的元数据:
 - LinksTo 关系中的 metadata.context: "电镀电镀参数参考"
- ④ AI 判断需要读取 Supplement_Methods.pdf
 - 文件为 PDF: AI 调用 PDF 解析能力, 提取完整制备步骤
 - 文件为 PPT: AI 提取幻灯片文本和图表描述
 - 文件为图片/视频: AI 作为多模态模型, 直接分析图像内容/视频帧
- ⑤ AI 综合 Knowledge.md + 附件内容 → 回答用户:
 - "制备流程分为三步: 1) SU-8 光刻定义电极图案;
 - 2) 磁控溅射 Au/Cr 种子层; 3) 电镀铂黑降低阻抗。
 - 详见 Supplement_Methods.pdf 第 3-5 页。"

场景二：用户点击链接直接跳转

- 用户阅读 文献1 的 Knowledge.md → 看到 "参考 [[任务1: 学习电镀铂黑]]"
- 按住 Ctrl/Cmd + 点击链接
- Workspace 切换到 任务1 的 Knowledge.md
- 用户可以立即查看任务1 的目标、进度、待办
- 任务1 底部的 Backlinks 自动显示 "文献1 链接了我"
- 用户可以从 任务1 跳回 文献1 (双向导航)

链接在查询中的角色总结：

环节	数据来源	作用
关键词搜索（精确匹配）	search.db（FTS5 全文索引）	找到包含"制备方法"文字的 Neuron
语义搜索（概念匹配）	vector.db（向量语义索引）	找到讨论"电极制备工艺"的 Neuron
链接关系增强（结构化关联）	graph.db LinksTo 关系	发现文献1 链接了 Supplement_Methods.pdf、任务1 链接了文献1
附件内容读取	Assets/ 原始文件（由链接告知路径）	AI 读取 PDF/图片/视频的原始内容
用户手动跳转	Knowledge.md 中的 <code>[[...]] / [text](path)</code>	用户按 Ctrl+Click 直接打开目标

核心结论： 链接不是替代 search.db/vector.db 的另一种索引，而是在它们的基础上提供了 **第四维度——结构化附件通路**。

- search.db 回答: "哪些文件提到了'制备方法'?"
- vector.db 回答: "哪些文件讨论的话题与'电极制备'相似?"
- graph.db LinksTo 回答: "文献1 明确指定了哪些附件是关键资源? 这些资源在哪里?"

当 AI 发现 Knowledge.md 不足以回答用户问题时, LinksTo 关系告诉 AI **应该去读哪个附件**——这就是 Neuro-Link 作为跨模态知识桥梁的核心价值。

1.9.4 多模态 AI 读取附件的能力

当 AI 通过链接（或直接扫描 Assets/ 目录）定位到附件后, Neuro 的 AI Engine 需要具备以下多模态读取能力：

附件类型	AI 读取方式	实现依赖
PDF（论文全文）	提取文本、图表标题、表格数据 → 纳入 AI 分析的上下文	PDF 解析引擎（如 PyMuPDF）+ AI 文本理解
图片（PNG/JPG/TIFF，如显微镜照片、实验装置图）	多模态模型直接分析图像内容（如 Claude/GPT-4V 的 vision 能力）	多模态 AI API
视频（MP4，如实验操作录像）	抽取关键帧 → 多模态模型逐帧分析 → 生成文字描述和时间轴标注	FFmpeg 帧抽取 + 多模态 AI API
表格（Excel/CSV，如阻抗测试数据）	解析表格结构 → AI 按列理解数据含义 → 生成统计摘要	表格解析库 + AI 数据分析
PPT/Keynote	提取幻灯片文本 + 图片 → AI 综合理解	文档解析库 + 多模态 AI
代码/脚本（如 MATLAB 分析脚本）	读取代码文本 → AI 理解分析逻辑和参数	AI 代码理解

Token 与成本策略：

多模态分析（尤其是视频帧分析和大量图片）的 Token 消耗远高于纯文本。Neuro 采用**按需加载**策略：

策略	说明
不自动全量分析	导入 Neuron 时只索引 Knowledge.md 和 metadata 文本，不自动分析附件内容
链接 = 分析优先级信号	出现在 Knowledge.md [text](Assets/...) 链接中的附件 → 高优先级，AI 主动提示用户"需要我分析附件吗？"
用户触发分析	用户可以在 Knowledge.md 的链接上右键 → "让 AI 分析此附件" → AI 开始多模态分析
分析结果回写	AI 分析附件后，结果写入 Knowledge.md 对应章节（如 # 怎么做 增补"详见 Supplement：制备步骤为..."），同时更新 search.db 和 vector.db 索引
缓存已分析结果	AI 分析过的附件摘要存入 .neuro/cache/summaries/，避免重复消耗 Token

具体流程举例（视频分析）：

- ① 文献1 的 Assets/ 中有 Supplement_Video.mp4（电镀铂黑的实验操作录像）
- ② 文献1 的 Knowledge.md 中写了：
"电镀过程详见 [实验操作录像](Assets/Supplement_Video.mp4)"
- ③ 用户右键链接 → "让 AI 分析此视频"
- ④ Neuro 后台：
FFmpeg 每隔 30 秒抽取一帧 → 得到 12 张关键帧图片
AI（多模态模型）逐帧分析 12 张图片
→ 生成描述："视频展示了铂黑电镀的完整流程：
0:00-1:30 准备铂黑溶液，将电极浸入
1:30-4:00 施加 10mA/cm² 电流密度，电镀 2 分钟
4:00-6:00 取出电极，去离子水冲洗，N₂ 吹干
6:00-7:00 显微镜下观察镀层均匀性"
- ⑤ 分析结果写入 Knowledge.md # 怎么做（或新增相应章节）
同时 graph.db 写入 LinksTo 关系（source: 文献1, target: Supplement_Video.mp4, type: LinksTo, metadata: {analysis: "已完成"}）
- ⑥ 下次用户 / AI 搜索 "铂黑电镀流程" 时：
search.db → 命中 文献1 Knowledge.md（已包含视频分析文字）
vector.db → 语义匹配
不再需要重复分析视频

1.10 Event System & Transaction（事件系统与事务）

四层架构描述了 Neuro 的静态结构。Event System 描述动态行为——当跨层操作发生时，如何保证一致性和可恢复性。

1.10.1 问题：跨层操作的级联效应

Neuro 的一个用户操作往往触发**跨越多层的连锁反应**。以拖拽 Neuron 改变归属 Space 为例：

用户拖拽 Paper001 从 Space_Electrode → Space_LIG

需要同步完成：

① metadata.json	→ knowledgeParents 字段更新	（事实层）
② search.db	→ FTS5 索引增量更新	（持久运行时）
③ vector.db	→ 向量标记过期，后台重建	（持久运行时）
④ graph.db	→ 如需更新 BelongTo 等关系	（持久运行时）
⑤ UI	→ Neuron Tree 重渲染	（表现层）

如果操作在①完成后、②执行时崩溃，会发生什么？

- metadata 已指向新 Space，但 search.db 的索引路径仍指向旧 Space
- 全文搜索可能返回错误的 Space 归属
- vector.db 的语义向量未标记过期，语义搜索结果基于旧数据

同样的问题存在于 **Neuron 创建**（metadata 写入 → search.db 索引 → vector.db embedding → graph.db 关系推荐）、**Knowledge.md 编辑**（文件保存 → search.db 增量索引 → vector.db 过期标记 → 反向链接更新 → cache 清理）等所有跨层操作。

1.10.2 方案：事件驱动的事务架构

核心思路：所有跨层操作通过 **事件总线（Event Bus）** 驱动。每个层的更新逻辑封装为独立的 **Listener**。事件从 Command 发出，Listener 链按序执行，支持失败回滚。

用户操作（UI）

↓

Command（MoveNeuronCommand）

↓ [事务边界开始]

Event: NeuronMoved { neuronId, fromParents, toParents, timestamp }

Listener 1: MetadataUpdater
→ metadata.json: knowledgeParents 更新

Listener 2: SearchIndexUpdater
→ search.db: FTS5 增量索引

Listener 3: VectorInvalidator
→ vector.db: 标记向量过期，入重建队列

Listener 4: GraphUpdater
→ graph.db: 更新 BelongTo 等导航相关关系

↓ [事务边界结束]

Listener 5: UIRefresher
→ Neuron Tree 重渲染（事务成功后异步进行）

这个模式与现代 IDE 的架构一致：

系统	事件示例	Listener 链
VS Code	<code>onDidChangeTextDocument</code>	→ Language Server → Diagnostics → Outline 刷新
JetBrains	<code>VirtualFileListener.contentsChanged</code>	→ PSI Tree 重建 → Inspection 重新运行 → Editor 标注更新
Neuro	<code>NeuronMoved</code>	→ Metadata → Search → Vector → Graph → UI

1.10.3 事件类型目录

所有跨层操作对应一个标准事件。以下是核心事件及其触发的 Listener 链：

事件	触发条件	Listener 执行链（事务范围内）
<code>NeuronCreated</code>	Connector 导入 / 用户手动创建	MetadataWriter → SearchIndexer → VectorEmbedder → GraphRelationRecommender
<code>NeuronMoved</code>	用户拖拽 Neuron 到新 Space	MetadataUpdater → SearchIndexUpdater → VectorInvalidator → GraphUpdater
<code>NeuronDeleted</code>	用户删除 Neuron	MetadataRemover → SearchIndexRemover → VectorRemover → GraphRelationCleaner
<code>KnowledgeEdited</code>	Knowledge.md 被用户/AI 编辑保存	SearchIncrementalIndexer → VectorExpiryMarker → BacklinkUpdater → CacheInvalidator
<code>MetadataEdited</code>	metadata.json 字段变更	SearchIncrementalIndexer → VectorExpiryMarker（如涉及 title/tags 等文本字段）
<code>RelationChanged</code>	graph.db 关系增/删/改	GraphCacheInvalidator → UIRefresher（Graph View）
<code>AssetAdded</code>	附件文件写入 Assets/	ThumbnailGenerator → SearchIndexer（如附件可提取文本）
<code>SpaceBehaviorChanged</code>	Space 的 behavior 字段更新	KnowledgeAggregationTrigger（如新增 KnowledgeAggregation 角色）

1.10.4 事务边界与回滚策略

事务范围： 仅覆盖数据层 Listener（metadata.json + search.db + vector.db + graph.db）。UI 刷新在事务提交后异步进行——UI 渲染失败不影响数据完整性。

失败处理流程：

Listener N 执行失败

↓

① 停止后续 Listener 执行

↓

② 执行回滚：逆序调用已完成 Listener 的 Rollback 方法

↓

③ 用户看到错误提示 + 操作未生效

↓

④ 事件日志写入 `.neuro/persist/event_log.db`（用于调试）

回滚机制——三层保障：

层	机制	说明
SQLite SAVEPOINT	search.db / vector.db / graph.db 在事务开始前创建 SAVEPOINT	失败时 ROLLBACK TO SAVEPOINT ，毫秒级恢复
应用层逆操作	metadata.json 变更前在内存中保存旧值	失败时写回旧值（metadata.json 是文件，不适用 SQLite SAVEPOINT）
事件日志	每次操作写入 .neuro/persist/event_log.db	崩溃恢复时重放日志，检查未完成的事务并补偿

崩溃恢复场景：

场景：Neuro 进程在 MoveNeuron 的 Listener 2 执行时被 kill

下次启动时：

- ① Neuro 读取 event_log.db，发现 Transaction T_xxx status = "pending"
- ② 检查各层实际状态：
 - metadata.json 是否已改？（对比 knowledgeParents 当前值 vs 事件记录的 toParents）
 - search.db 索引是否已更新？
- ③ 判定策略：
 - 若 metadata 已改 → 从 Listener 2 继续补执行（前向恢复）
 - 若 metadata 未改 → 事务未生效，清理日志即可
 - 若部分 Listener 完成 → 回滚已完成部分 + 重新执行（或前向补完）
- ④ 恢复完成后标记 Transaction status = "resolved"

1.10.5 幂等性设计

每个 Listener 的操作必须设计为**幂等**——重复执行不产生副作用：

Listener	幂等保证方式
MetadataUpdater	写入前检查当前值：若 knowledgeParents 已包含目标 Space ID，跳过
SearchIndexUpdater	FTS5 的 INSERT OR REPLACE 语义天然幂等
VectorInvalidator	标记过期是状态设置（而非追加），重复标记结果相同
GraphUpdater	INSERT OR IGNORE / UPDATE 语义

1.10.6 并发控制

策略	说明
Per-Neuron 串行化	同一 Neuron 的操作排队执行（per-Neuron in-memory queue），避免 metadata.json 文件级写冲突
不同 Neuron 可并行	Neuron A 的 Move 和 Neuron B 的 Create 可同时进行，各自独立的事务
读写分离	读操作（View 查询）不阻塞写事务，直接读取当前已提交状态

1.10.7 与 Knowledge Lifecycle 的关系

第四章 定义的七个宏观阶段（Connector → Create → Understand → Grow → Connect → Work → Evolve）是**时间的维度**——描述一个 Neuron 从无到有、从简单到丰富的成长故事。

Event System 是**操作的维度**——描述用户或 AI 每次交互时，Neuro 各层如何协同、如何保证一致性。

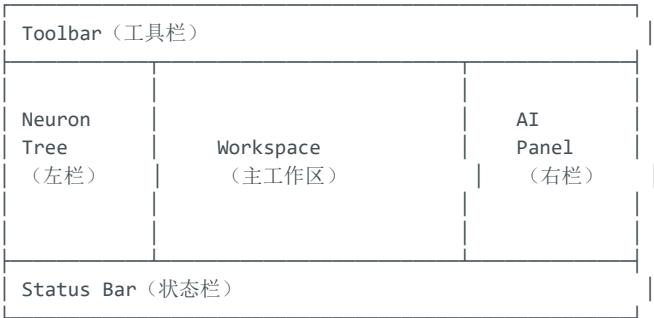
两者的关系：

Knowledge Lifecycle（宏观）		
Create 阶段		
└─ NeuronCreated 事件（微观事务）	← 一次性	
└─ KnowledgeEdited 事件	← AI 写 Knowledge.md 初版	
└─ RelationChanged 事件	← AI 推荐初始关系	
Grow 阶段		
└─ KnowledgeEdited 事件 × N	← AI 持续完善 Knowledge.md	
└─ MetadataEdited 事件 × M	← AI 自动打标签、更新状态	
└─ RelationChanged 事件 × K	← AI 建立更多关联	
Evolve 阶段		
└─ NeuronMoved 事件	← 用户/AI 重新归类	
└─ KnowledgeEdited 事件 × ...	← 知识持续演化	

宏观生命周期由一系列微观事务事件组成。每个阶段都通过事件总线驱动，每个事件都受事务保护。

第二章 工作区架构（Workspace Architecture）

2.1 整体布局



2.2 Neuron Tree（左栏）

不再叫 Explorer 或文件树或 Knowledge Tree。

Neuron Tree 展示的是**知识投影网络（Knowledge Projection）**，不是文件目录树。它基于 metadata.json 中的 `knowledgeParents` 字段渲染，通过 DFS 实时构建完整树，同一 Neuron 可以在多个逻辑路径下同时出现。

2.3 Workspace（中栏）

主工作区，根据当前选中的 View 类型展示不同内容。

2.4 AI Panel（右栏）

统一 AI 交互入口。所有 AI 交互都在右栏完成。

2.5 Toolbar（顶部工具栏）

- 项目切换
- View 类型切换
- 搜索

- 插件管理
- 全局设置

2.6 Status Bar（底部状态栏）

- 当前 Neuron 数量
- AI 状态（分析中 / 就绪）
- 同步状态

第三章 数据组织方式（Data Architecture）

3.1 统一数据模型

所有 Neuron 遵循同一数据模型：

```
Neuron_ID/  
├─ metadata.json    ← 事实层  
├─ Knowledge.md     ← 知识层  
└─ Assets/          ← 资源层  
    ├─ paper.pdf  
    ├─ Figure1.png  
    └─ Supplement.zip
```

说明： 不存在每个 Neuron 级别的运行时目录。所有运行时数据统一由项目级 `.neuro/` 管理。

3.2 四层架构（Four-layer Architecture）

Neuron 的数据从概念上分为四层，但其中两层存储在 Neuron 目录中，另外两层在 `.neuro/ runtime` 中：

层级	存储位置	文件	核心职责	是否 允许 AI 修 改	是否 允许 用户 修改	生 命 周 期
事实层 (Fact Layer)	Neuron 目录	<code>metadata.json</code>	记录结构化客观事实：类型、kind、标题、作者、DOI、标签、physicalPath、knowledgeParents、schemaVersion、capabilities	极少 （仅修正事实）	可以	基本稳定
知识层 (Knowledge Layer)	Neuron 目录	<code>Knowledge.md</code>	AI 持续维护的活知识索引：摘要、关键发现、AI 分析、关联列表	持续更新	可以（协同编辑）	持续成长
关系层 (Relation Layer)	<code>.neuro/graph.db</code>	关系数据库	记录 Neuron 之间的所有关联：论文归属课题组、同项目版本关系、课题组间合作网络、引用、依赖等。这是支撑世界地图课题组可视	持续更新	间接（通过 UI）	持续成长

层级	存储位置	文件	核心职责	是否允许 AI 修改	是否允许用户修改	生命周期
			化、合作网络图谱的数据基础			
语义层 (Semantic Layer)	.neuro/vector.db	向量数据库	将 Knowledge.md 和 metadata 文本转化为向量嵌入，供 AI 语义检索和关系推荐使用。用户不直接操作，AI 在 RAG 和关系推荐时自动读写	自动生成	不直接编辑	自动维护

这四层架构是 Neuro 数据模型的核心架构，确保职责清晰、互不混淆。

3.3 物理目录结构（统一方案）

设计决定：所有 Neuron 统一按类型分目录存放，不再支持 Units/ 平铺方案。Neuron 目录使用有意义命名（如 Paper_Flexible_LIG_Electrode_2025），而非无意义 ID（如 KU_000001）。

```
研究生/
├── metadata.json
├── Knowledge.md
├── Assets/
├── 文献库/
│   ├── 脑类器官/
│   │   ├── 脑类器官神经电极/
│   │   │   ├── 文献1_StretchableMesh/
│   │   │   │   ├── metadata.json
│   │   │   │   ├── Knowledge.md
│   │   │   │   └── Assets/
│   │   └── 文献2_RapidPrototyping/
│   └── 任务管理/
│       ├── 脑类器官神经电极设计与制备/
│       │   ├── 任务1_学习电镀铂黑/
│       │   └── 任务2_制备柔性神经电极/
│       └── 实验记录/
│           ├── 实验1_LIG功率矩阵/
│           └── 实验2_阻抗验证/
│       └── 课题组与机构/
│           ├── Harvard_Liu_Lab/
│           └── MIT_Media_Lab/
│       └── 人员/
│           └── Liu_Jia/
└── .neuro/
```

← 项目根目录（也是一个 Neuron, type: project）
 ← Project 自己的 metadata
 ← Project 级别的知识总结
 ← type: space（一级子文件夹）
 ← type: space（二级子文件夹）
 ← type: space（三级子文件夹）
 ← type: paper
 ← type: paper
 ← type: space（一级子文件夹）
 ← type: space（二级子文件夹）
 ← type: task
 ← type: task
 ← type: space
 ← type: experiment
 ← type: experiment
 ← type: space（网页/课题组/机构页面集合）
 ← type: group（课题组是 leaf Neuron, 不是 Space）
 ← type: group
 ← type: space（研究人员集合）
 ← type: person
 ← Neuro Runtime（不进 Git, persist/ + cache/ 结构见 1.6 节）

关键原则：

- 1. 每个叶子节点都是完整的 Neuron——无论它是 paper、task、space 还是 person
- 2. Space (type: space) 构成中间节点——文献库/、脑类器官/、脑类器官神经电极/、任务管理/、脑类器官神经电极设计与制备/ 等均为 Knowledge Space
- 3. AI 不依赖目录结构——AI 只读 metadata.json 中的 type、knowledgeParents 等字段，目录结构仅为 用户视觉服务
- 4. 目录服务于用户的操作习惯，分类结构不成为知识模型本身

5. Space 的层级深度由 **knowledgeParents** 组成的树状结构隐式表达，无需 **level** 字段区分一级/二级/三级 Space。详见 1.5 节的层级讨论

3.4 Neuron Tree 渲染规则

Neuro 左侧的 Neuron Tree 展示的是 **knowledgeParents** 经 DFS 构建的完整树，不是 **physicalPath**。

渲染机制： 渲染器收集项目中所有 Neuron（Space + 叶子）的 **knowledgeParents** 取并集，通过 DFS 实时构建完整树。

Neuron Tree 中的显示（完整数据源，以 1.5 节文件树为例）：

- 文献库
 - 脑类器官
 - 脑类器官神经电极
 - 文献1: Stretchable Mesh Nanoelectronics...
 - 文献2: Rapid prototyping of a 3D...
- 任务管理
 - 脑类器官神经电极设计与制备
 - 文献1: Stretchable Mesh Nanoelectronics... ← 同一 Neuron 的二次投影
 - 任务1: 学习电镀铂黑
 - 任务2: 制备柔性神经电极

同一个 Neuron 可以出现在多个 knowledge path 下，但硬盘上只有一份数据。

knowledgeParents 的深层本质：一种可定制的上下文关系

knowledgeParents 本质上也是一种描述 Neuron 之间关系的工具。Space、Project 与 Paper、Task 之间的归属关系，就是通过 knowledgeParents 表达的。因此知识的结构不只是 graph.db 中的 Relation，也反映在用户每天面对的 Neuron Tree 中。

这意味着用户可以根据自己的需求选择不同的'路径模板'来切换 Neuron Tree 的呈现形式，而无需修改物理数据：

路径模板	数据来源	呈现形式	适用场景
默认（按类型）	knowledgeParents（DFS 构建）	Project → Paper → Space → Sub-Space → Paper_A	日常科研管理
课题组学术地图	graph.db（BelongTo 关系）	Project → Country → Group → Paper_A	全球课题组分布与合作网络
期刊投稿分析	metadata.journal + year 字段	Project → Journal → Year → Paper_A	研究目标期刊发文风格
时间线	metadata.year 字段	Project → 2025 → 2026 → Paper_A	按年份浏览产出

实现机制：

- knowledgeParents 在 metadata.json 中始终存储父 Space 的 ID 列表（如 ["Space_BrainOrganoid_Electrode", "Space_LIG"]）
- 默认模板从所有 Neuron 的 knowledgeParents 取并集，通过 DFS 实时构建树
- 其他模板的数据来源不同（graph.db 关系 / metadata 字段），渲染器根据模板类型选择不同数据源
- 用户可在 View 设置中切换路径模板——切换只改变渲染逻辑，不改变 metadata.json
- 高级用户可创建自定义路径模板（如按基金号、按实验方法）
- 切换路径模板不消耗 Token（纯本地计算），仅当 AI 需要根据用户意图自动推荐模板时才涉及 AI 调用
- 非默认模板依赖数据准备：课题组地图需要 graph.db 中已有 BelongTo 关系，期刊分析需要 metadata 中的 journal 字段非空

设计原则： `knowledgeParents` 是灵活的关系描述工具，不是固定编码的路径。它的核心职责是呈现用户意图——我想让这个 `Neuron` 出现在树中的哪个位置。而 `graph.db` 的核心职责是呈现知识关联——这些 `Neuron` 实际存在什么关系。两者彻底分离，共同支撑不同的视图模板。

第四章 Knowledge 生命周期（Knowledge Lifecycle）

这是 `Neuro` 最大的创新点，也是整个产品的灵魂。

传统工具：导入 → 保存 → 结束。

`Neuro`：知识是活的，它在不断成长。

- ① Connector（采集）
浏览器插件 / `Zotero` 同步 / 手动上传
↓
- ② Create（创建）
`Neuron` 初始化, `metadata.json` 写入, `Assets` 归档
🟢 `search.db` 立即索引 `metadata` 文本字段（毫秒级，同步）
↓
- ③ Understand（理解）
`AI` 读取知识内容，生成 `Knowledge.md` 初始版本
🟢 `search.db` 增量索引 `Knowledge.md` 全文
🟠 `vector.db` 异步计算 `Embedding` 向量（后台队列，不阻塞 UI）
↓
- ④ Grow（成长）
`AI` 持续完善 `Knowledge.md` 五章内容：
- 充实 # 总结 → 提炼更精确的 `Summary`
- 更新 # 怎么做 → 提取关键参数和方法细节
- 丰富 # 思考 → 生成 `AI Insights`，深化分析
- 补充 # 经验 → 建立跨 `Neuron` 关联（写入 `graph.db LinksTo` 关系）
`AI` 同步更新 `metadata` 字段：
- 自动打标签（`metadata.tags`）→ `search.db` 增量更新 + `vector.db` 标记过期
- 更新状态（`metadata.status`）
↓
↓
- ⑤ Connect（连接）
建立与其他 `Neuron` 之间的 `Relation`（存入 `.neuro/graph.db`）
用户可在 `Graph View` 中查看关联网络
↓
- ⑥ Work（工作）
各种 `View` 围绕 `Neuron` 协同工作：
- `Graph` 中探索知识关联
- `Kanban` 中跟踪任务进度
- `Timeline` 中查看时间线
↓
- ⑦ Evolve（演化）—— 知识培育（`Knowledge Cultivation`）

这不是一种"模式"，而是 `Neuro` 结合 `AI` 管理知识的核心功能之一。

用户根据模板创建任务管理项目并接入 `AI` 后，`AI` 引导用户完整设计项目计划（或用户根据模板自行填写 `task` 信息）：
- 形成任务名称/负责人/时间/前置任务等信息
- 该信息存储在 `task Neuron` 的 `Knowledge.md` 和 `metadata.json` 中

`AI` 进行总结并可视化呈现为甘特图或任务流程图（用户也可自主选择手动编辑）：
- 拖动甘特图上的任务时间条 → `.md` 文件实时更新
- 修改任务状态/负责人 → `metadata.json` 同步更新
- 可视化界面与数据存储之间保持实时双向同步

形成完整的任务管理系统后，`AI` 开始**事件驱动**的跟进：
- 用户向 `AI` 汇报工作内容 → `AI` 分析并更新 `task` 相关信息
- `Knowledge.md` 持续更新，可视化同步更新
- `AI` 根据当前进度给出下一步建议

- 用户与 AI 的交互是主动触发（用户告知），而非 AI 轮询

后期将开发更强的检测能力（规划中）：

- **桌面 AI 助手**：捕获用户屏幕信息，分析用户当前行为，给出任务建议或提醒下一步
- **实体桌面宠物**（长期规划）：安装摄像头与语音模块，甚至接入全屋智能
 - 在用户久坐时提醒起身运动
 - 在用户长时间停留一个页面思考时给出相关建议

Neuro 不保存知识，Neuro 培养知识。

关于每个阶段如何保证数据一致性： 生命周期中的每一步——从 Connector 创建 Neuron 到 AI 更新 Knowledge.md——都通过 [1.10 节 Event System & Transaction](#) 中定义的事件驱动事务架构执行。宏观阶段由一系列微观事务事件组成，每个事件都受三层回滚保障（SQLite SAVEPOINT + 逆操作 + event_log.db）。

第五章 AI 引擎（AI Engine）

5.1 AI 的核心工作方式

AI 始终围绕 **Neuron** 工作，不是围绕 PDF，不是围绕文件。

AI 访问策略：

1. **本地优先：** AI 首先读取 Knowledge.md（每个 Neuron 的知识索引）
2. **按需扩展：** 如需更多信息，通过 Assets/ 按需读取。Knowledge.md 中的 `[text](Assets/...)` 链接提供了结构化的附件发现路径；graph.db 中的 `LinksTo` 关系提供了跨 Neuron 的资源关联（详见 1.9 节 Neuro-Link）
3. **多模态感知：** AI 可直接分析图片（显微镜照片、实验装置图）、视频帧（实验操作录像）、PDF 全文、表格数据等附件内容，而非仅限文本（详见 1.9.4 节）
4. **Token 优化：** Knowledge.md 本身就是知识精华，无需频繁读取 PDF
5. **用户可控：** 用户可指定 AI 分析的深度与范围

5.2 AI 能力矩阵

Understand（理解）

读取 Knowledge.md，理解 Neuron 内容与上下文

Extract（提取）

从附件（PDF/网页/视频）中提取关键信息、参数、数据

Perceive（多模态感知）

直接分析图像、视频帧等非文本内容

通过 Knowledge.md 中的链接或 graph.db LinksTo 关系发现附件

对图片进行视觉理解，对视频逐帧分析并生成文字描述（详见 1.9.4 节）

Summarize（摘要）

生成摘要、核心观点、关键结论

Tag（打标）

自动生成标签，更新 metadata.json 中的 tags

Reason（推理）

跨 Neuron 的推理分析：对比、关联、推断

结果写入 .neuro/graph.db（关系）和 Knowledge.md（分析结论）

Generate（生成）

更新 Knowledge.md、生成可视化、生成报告

Observe（观察）

事件驱动：用户主动向 AI 汇报进度 → AI 分析并更新 task 的 Knowledge.md

可视化（甘特图/任务流程图）同步实时更新
AI 根据当前进度给出下一步建议
后期规划：桌面 AI 助手/实体桌面宠物辅助检测（需严格隐私授权）

Reflect（反思）
根据用户反馈和新的知识输入，主动优化已有 Knowledge.md

5.3 AI 接入方式

- 支持多种 AI 模型（Claude / GPT / Gemini 等）
- 所有 AI 通过统一的 AI Engine 接口调用
- 支持本地模型离线运行（未来）

第六章 关联系统（Relation System）

6.1 关系类型

关系类型	语义	示例
引用（Cite）	A 引用 B	论文1 引用 论文2
支持（Support）	A 支持 B	实验数据 支持 研究结论
产生（Produce）	A 产生 B	实验 产生 结果
触发（Trigger）	A 触发 B	任务完成 触发 新任务
属于（BelongTo）	A 属于 B	论文 属于 Harvard_Liu_Lab Space
依赖（DependsOn）	A 依赖 B	任务B 依赖 任务A
关联（RelatedTo）	A 与 B 相关	网页攻略1 关联 网页攻略2
同项目（SameProject）	A 与 B 同项目	两篇论文是同一科研项目的原始版与修改版，metadata 中记录版本说明
合作（CollaboratesWith）	A 与 B 合作	Harvard_Liu_Lab 与 MIT_Media_Lab 合作发表 N 篇论文，weight 记录次数
链接（LinksTo）	A 链接到 B	文献1 的 Knowledge.md 中写了 [详见附件](Assets/Supp.pdf) 或 [[任务1]] → graph.db 自动写入 LinksTo 关系

6.2 关系的数据存储

关系不再存储在 Neuron 目录的 JSON 文件中，而是存储在 `.neuro/graph.db`（SQLite 数据库）中。

`.neuro/graph.db`

表: relations


```
├── source_id      → Neuron ID
├── target_id      → Neuron ID
├── relation_type  → cite / support / produce / trigger /
                  belong_to / depends_on / related_to /
                  same_project / collaborates_with /
                  links_to
├── weight         → 关系强度 (AI 自动评估)
├── created_at
└── metadata      → JSON 额外信息
```

为什么用数据库而非 Markdown/JSON?

- Graph 每天查询"某 Neuron 有哪些邻居"——数据库 0.001 秒，文件遍历数十万个文件慢得多
- 关系量级可达数十万条，不适合文件系统管理

6.3 关系推荐

- AI 根据 Knowledge.md 和 metadata 自动推荐潜在关系
- 推荐的关系写入 `.neuro/graph.db` 并标记为"待确认"
- 用户可手动确认或拒绝

关系触发策略

触发时机	触发条件	推荐的典型关系
Knowledge.md 保存	用户/AI 在 Knowledge.md 中写入 <code>[[...]]</code> 或 <code>[text](Assets/...)</code> 链接	Markdown 解析器自动检测链接语法 → graph.db 写入 <code>LinksTo</code> 关系（无需 AI 参与，纯解析器行为）
新 Neuron 创建时	Connector 导入/手动创建 Neuron	AI 自动检测与已有 Neuron 的引用、关联关系
批量导入后	一次性导入多篇论文/多个实体	同作者归属、同课题组关联、同项目版本
用户显式触发	用户在 Space 右键选择"推荐关系"	全量扫描该 Space 下的所有 Neuron
AI 定时巡检	每日/每周后台分析（可配置频率）	发现新关联、更新关系权重

性能策略： 全量推荐是计算密集型操作（需读取所有 Knowledge.md 进行语义分析），Token 消耗由用户承担。因此系统默认仅在"新 Neuron 创建时"做增量推荐，全量推荐需用户显式触发或配置定时任务。

知识图谱驱动的研究课题组可视化（展望）

基于 `.neuro/graph.db` 中的 `BelongTo`、`SameProject`、`CollaboratesWith` 关系，Neuro 可提供研究课题组地图视图：

1. **世界地图层：** 查询 `type: group` 且含地理位置的 Neuron 的 `metadata.json`，读取经纬度，在地图上渲染为点
2. **气泡大小与颜色：** `metadata.json` 中记录 `paperCount`、`citationCount`，决定点的大小和颜色深浅
3. **点击展开：** 点击课题组气泡，查询 graph.db 中所有 `BelongTo → this_group` 的 Neuron（论文列表）
4. **合作连线：** 查询 `CollaboratesWith` 关系，点之间绘制连线，`weight` 字段决定连线粗细
5. **导师信息：** 查询 `BelongTo → this_group` 的 `type: person` Neuron，在课题组详情页展示

注意： 课题组以 `type: group` 的 leaf Neuron 而非 Space 存储。论文与课题组的关系通过 `knowledgeParents`（用户导航用）和 `graph.db`（知识关联用）双重表达。论文属于多个课题组时，graph.db 中的多条 `BelongTo` 关系即可表达，不增加目录复杂度。

此视图作为**插件化 View 类型**开发，纳入 v2.0 开放平台路线。

第七章 视图系统（View System）

7.1 统一视图框架

不再有"Entry 模式""Thinking 模式""Print 模式""Follow 模式"（这些从来都不是"模式"，而是 Neuro 的功能/视图，现在统一为 View 和知识培育功能）。

所有可视化呈现统一称为 **View**，用户通过 View 切换器在不同视图之间切换。

7.2 内置视图

视图	功能	适用场景
Markdown View	完整渲染 Knowledge.md，支持编辑	阅读与编辑 Neuron
Tree View	展示知识空间投影层级	浏览项目结构
Graph View	节点-边形式展示 Neuron 关联网络	知识关联探索
Timeline View	按时间线排列 Neuron	研究历程、旅行计划
Kanban View	看板展示任务状态	任务管理
Calendar View	日历展示任务与事件	日程管理
Table View	表格化显示 Neuron 及关键字段	批量浏览与筛选
Gallery View	卡片墙形式展示	视觉化浏览
Mind Map View	思维导图形式	头脑风暴与知识拆解

7.3 预留接口

- 开放插件接口，允许社区开发新的 View 类型
- 用户可自定义 View 的布局与样式

第八章 模板系统（Template System）

8.1 模板的作用

用户创建一个 Neuron 时，选择 Template 来确定其结构和初始内容。

8.2 内置模板

注：所有模板的 Knowledge.md 均采用统一的五章结构（总结 / 为什么 / 怎么做 / 思考 / 经验）。
metadata 字段（Title / Tags / Status 等）在 metadata.json 中定义，不写入 Knowledge.md。下表列出各 type 在五章中的填充侧重。

模板类型	五章填充侧重	适用场景
Paper Template	总结=研究发现 / 为什么=研究动机 / 怎么做=方法与参数 / 思考=AI 分析 / 经验=教训与关联文献	学术文献
Experiment Template	总结=实验结果 / 为什么=实验目的 / 怎么做=材料与方法 / 思考=分析与推断 / 经验=可复用流程	科研实验
Task Template	总结=任务目标 / 为什么=背景动机 / 怎么做=Todo List / 思考=AI 建议 / 经验=风险与教训	任务管理
Webpage Template	总结=核心观点 / 为什么=浏览背景 / 怎么做=内容摘要 / 思考=分析评价 / 经验=相关资源	网页信息保存
Video Template	总结=视频主题 / 为什么=观看背景 / 怎么做=关键帧/步骤 / 思考=AI 分析 / 经验=相关资源	视频资料
Meeting Template	总结=会议结论 / 为什么=会议背景 / 怎么做=议程要点 / 思考=洞察分析 / 经验=待办与后续行动	会议记录
Note Template	总结=核心内容 / 为什么=记录背景 / 怎么做=详细内容 / 思考=个人分析 / 经验=关联启发	自由笔记
Space Template	总结=领域概览 / 为什么=领域意义 / 怎么做=研究路径 / 思考=趋势热点 / 经验=推荐阅读	知识空间
Group Template	总结=课题组概况 / 为什么=成立背景 / 怎么做=研究方向 / 思考=领域影响力 / 经验=代表性成果	课题组/实验室

8.3 模板的可扩展性

- 用户可自定义模板
- 开源社区可通过插件贡献新模板
- AI 可根据内容推荐模板

第九章 插件架构（Plugin Architecture）

9.1 插件化设计原则

Connector 不是系统功能，是 Plugin。

Zotero 集成不是系统功能，是 Plugin。

Obsidian 同步不是系统功能，是 Plugin。

外部集成全部以 Plugin 形式接入，不写死在系统核心中。

9.2 内置插件

插件类型	功能	备注
Neuro Connector	浏览器插件，一键采集网页/文献/视频	核心插件，默认安装
Zotero Sync	从 Zotero 导入条目	可选
Obsidian Sync	与 Obsidian Vault 同步	可选

插件类型	功能	备注
Notion Sync	与 Notion 数据库同步	可选
Claude Code Integration	Claude Code 命令行集成	可选
Git Integration	Git 版本控制集成	可选

9.3 Connector 插件功能

支持：Chrome / Firefox / Edge

核心功能：

- 自动识别页面类型（学术文献/网页/视频）
- 一键抓取标题、URL、摘要、封面图
- 自动下载附件（PDF 等）
- 保存时允许编辑名称、标签、简记
- 自动归类到指定 Knowledge Space

第十章 扩展能力（Extension）

10.1 基础扩展

- Template**：自定义模板
- Workflow**：自定义工作流（如"导入论文 → AI 分析 → 生成报告"）
- Automation**：自动化规则（如"当新论文导入时，自动打标签并归类"）

10.2 开发者扩展

- API**：RESTful API，允许第三程序读写 Neuron
- SDK**：开发者工具包，用于构建插件
- MCP**：Model Context Protocol 支持，允许 AI 直接操作 Neuro 数据

第十一章 推荐产品路线（MVP 策略）

基于与 ChatGPT 的深入讨论，推荐以下三阶段路线：

第一阶段：Neuro Runtime（MVP）

目标：充分利用现有生态，构建统一数据模型，以最低成本验证核心理念。

- 底层调用 Zotero（文献管理）、Obsidian（Markdown 编辑）、Git（版本控制）、Claude Code（AI 能力）
- Neuro 只负责：**统一数据模型（Neuron） + Knowledge.md + Relation + AI 工作流编排**
- 核心交付：**.neuro/** Runtime + Neuron 数据结构 + 基本 AI 交互

第二阶段：原生模块替换

目标： 逐步将最有价值的模块替换为原生实现。

- Neuron Tree（取代文件树）
- Knowledge Lifecycle（Grow + Connect + Evolve）
- Knowledge Cultivation（知识培育，事件驱动的 AI 跟进）

第三阶段：一体化桌面应用

目标： 当产品成熟后，构建真正的 AI Native 桌面应用。

第三部分：PRD 功能概述（基于 PAD 落地的功能清单）

以下功能清单是 PAD 的实现。每个功能的详细 PRD（含用户故事、交互流程、异常处理、验收标准）将在后续独立文档中展开。

F1 Connector（浏览器插件）

- **目标：** 实现从浏览器到 Neuro 的一键知识采集
- **核心流程：** 浏览网页 → 点击插件 → 识别类型 → 抓取元数据 → 保存到指定 Project
- **用户故事：** 研究员在 arXiv 上看到一篇论文，点击 Neuro Connector，自动保存 PDF 和元数据到"脑类器官电极"Space

F2 Neuron Tree（神经元树）

- **目标：** 实现按 `knowledgeParents` DFS 构建的知识导航树
- **核心流程：** 用户在左栏浏览 Neuron Tree → 点击 Neuron → Workspace 中打开
- **用户故事：** 用户在 Neuron Tree 中看到"Brain Organoid/Electrode"下所有相关论文，其中一篇同时出现在"LIG"路径下

F3 Neuron 管理

- **目标：** Neuron 的创建、编辑、删除（分类删除 vs Neuron 删除）
- **核心流程：** 用户选择 Template → 创建 Neuron → AI 自动初始化 Knowledge.md
- **用户故事：** 右键 Neuron，选择"删除分类"（仅移除 knowledgePath）或"删除 Neuron"（删除整个数据）

F4 AI Engine

- **目标：** 围绕 Neuron 提供 AI 理解、提取、标注、推理
- **核心流程：** 用户选择 Neuron → 触发 AI → AI 读取 Knowledge.md → AI 更新 Knowledge.md + 写入 `.neuro/`
- **用户故事：** 用户导入新论文，AI 自动提取关键参数（电极材料、阻抗）写入 Knowledge.md，同时建立与已有论文的关联（写入 graph.db）

F5 AI Chat

- 目标：基于当前项目上下文的自然语言对话
- 核心流程：用户提问 → AI 优先检索 Knowledge.md → AI 结合 `.neuro/vector.db` RAG → 生成回答
- 用户故事：用户提问"这个项目中阻抗最低的电极材料是什么？"，AI 检索所有 Paper 的 Knowledge.md 并给出对比表格

F6 View System

- 目标：提供 Markdown / Graph / Timeline / Kanban / Calendar 等多种视图
- 核心流程：用户切换 View → Workspace 切换为对应视图

F7 知识培育（Knowledge Cultivation）

这不是一种"模式"，而是 Neuro 结合 AI 管理知识的核心功能。

- 目标：AI 事件驱动地跟踪项目进展，自动更新任务信息和可视化呈现
- 核心流程：
 - 用户根据模板创建任务管理项目 → AI 引导/用户自行设计项目计划
 - AI 可视化呈现为甘特图或任务流程图（可拖拽编辑）
 - 可视化编辑实时同步到 metadata.json 和 Knowledge.md
 - 用户主动向 AI 汇报进度 → AI 分析并更新 task 信息 → 可视化同步更新
 - AI 根据当前进度给出下一步建议
- 用户故事：用户创建"制作 LIG 电极"项目，AI 生成甘特图。用户拖动时间条调整任务排期，对应 task 的 metadata.json 实时更新。完成实验后告诉 AI"阻抗测试完成，结果为 5kΩ"，AI 更新 Knowledge.md 和 graph.db，甘特图上该任务自动标记完成

F8 Plugin System

- 目标：开放的插件架构
- 核心流程：用户安装 Plugin → Plugin 注册到系统 → 用户使用功能

第四部分：技术架构概要

详细技术设计将在后续 Technical Design 文档中展开。

数据存储体系

层级	技术方案	说明
文件系统	本地目录	存储 Neuron 目录（metadata.json + Knowledge.md + Assets/）
运行时数据库（持久）	SQLite (<code>.neuro/persist/</code>)	graph.db（关系）、vector.db（向量）、search.db（全文）

层级	技术方案	说明
运行时缓存（可丢弃）	文件/SQLite (.neuro/cache/)	history.db（版本历史）、summaries/（摘要缓存）、thumbnails/（缩略图）
向量检索	SQLite + 向量插件 / FAISS	AI 检索增强（RAG），集成在 .neuro/persist/vector.db
Knowledge.md	Markdown 文件	AI 可读/写的人类可读知识索引

前端技术

- 桌面应用：Electron（跨平台）
- UI 框架：React / Vue（待定）
- 视图渲染：内置 Markdown 解析器 + D3.js（Graph 视图）+ 第三方图表库

后端技术

- 本地服务：Node.js / Python（可选）
- AI 接口：Claude API / OpenAI API / 国产大模型 API
- 插件系统：Plugin SDK + MCP 协议

第五部分：版本规划

v0.1 Alpha（数据模型验证 + 最小可用 UI）

目标： 验证 Neuron 数据模型和基础 Runtime，提供最小可交互界面

这个阶段用户可能感觉"就像在用 Obsidian"，这是正常的。我们不追求一步到位，后续版本持续优化。

最小 Electron 窗口：



- ☐ Neuron 数据结构实现（metadata.json + Knowledge.md + Assets/）
- ☐ [.neuro/](#) Runtime 目录初始化（persist/ + cache/ 分离）
- ☐ graph.db 关系数据库基础实现
- ☐ Neuron Tree 基本渲染（基于 knowledgeParents DFS）
- ☐ Markdown View（Knowledge.md 渲染与编辑）

v0.2 Beta (Connector + AI)

目标：实现外部数据采集和 AI 基础分析

- ☐ Neuro Connector 浏览器插件（基础版）
- ☐ AI Engine：Understand + Summarize + Tag
- ☐ AI Chat（项目上下文对话）
- ☐ Paper / Task / Space Template

v1.0 正式版 (View + Graph)

目标：核心可视化和知识关联

- ☐ Graph View（从 graph.db 读取）
- ☐ Timeline View
- ☐ Kanban View
- ☐ Calendar View
- ☐ Table View（metadata.json 字段映射）
- ☐ Relation System 完整实现 + AI 推荐
- ☐ Template 自定义

v1.5 (知识培育 Knowledge Cultivation)

目标：AI 事件驱动的知识演化和任务跟踪

- ☐ 知识培育功能：AI 事件驱动跟进 + 可视化（甘特图/任务流程图）
- ☐ 可视化编辑实时双向同步（甘特图 ↔ metadata.json / Knowledge.md）
- ☐ AI 自动更新 Knowledge.md + graph.db
- ☐ history.db 版本管理
- ☐ vector.db 语义检索

v2.0 (开放平台)

目标：插件生态和开发者工具

- ☐ Plugin SDK
- ☐ RESTful API
- ☐ MCP 协议支持
- ☐ 社区插件市场
- ☐ Zotero / Obsidian / Notion 同步插件

附录：概念升级对照表

原始概念	最终定位	升级说明
Entry / 主节点 / Knowledge Unit	Neuron	统一为神经元模型，所有知识对象共享同一数据结构。 ID 改为有意义命名，不再使用KU_ 前缀

原始概念	最终定位	升级说明
Collection / 子级分类	Knowledge Space (type: space)	目录节点升级为"知识空间", 具备 AI 综述能力。层级由 knowledgeParents (父 Space ID) 经 DFS 实时构建
Explorer / 文件树	Neuron Tree	基于 knowledgeParents DFS 渲染, 非物理目录树
主节点 Markdown / Knowledge.md	Knowledge.md (Living Knowledge)	明确定义为 AI 持续维护的活知识索引
metadata	metadata.json (Fact Layer)	新增 physicalPath (相对路径)、knowledgeParents (ID 列表)、kind、schemaVersion、capabilities
Thinking 模式	AI Engine 能力矩阵	抽象概念统一为具体的 AI 能力
Print 模式	View System	所有可视化统一称为 View
Follow 模式	知识培育 (Knowledge Cultivation, 事件驱动)	从被动"模式"升级为主动培育功能, 不再称"模式"
Relation 文件	.neuro/graph.db	从 Neuron 目录级 JSON 升级为项目级图数据库。仅存 Entity Relation, 不存 Navigation。新增 SameProject / CollaboratesWith 类型
项目管理/任务	Task Neuron (type: task)	任务与论文、实验平级, 都是 Neuron。type + kind 双层分类
标签系统	metadata.tags + knowledgeParents	标签用于搜索, knowledgeParents 用于导航
AI 对话	AI Panel	所有 AI 交互集中在右侧面板
插件系统	Plugin Architecture	Connector/Zotero/Obsidian 全部降级为插件
—	四层架构	新增 (Fact → Knowledge → Relation → Semantic)
—	.neuro/ Runtime	新增 (类比 .git/, 管理运行时数据库和缓存)
—	物理/逻辑分离	新增 (physicalPath vs knowledgeParents)
—	type + kind 双层分类	新增, type 为顶层类型 (paper/task/space), kind 为子类型 (research/review/patent; todo/bug/milestone; domain/category)
—	schemaVersion	新增 metadata 字段, 支持未来数据模型版本迁移
—	Manifest (capabilities)	新增, 声明 Neuron 的能力 (Editable/Runnable/Playable), 替代基于 type 的判断逻辑
—	Space Behavior	新增, Space 可声明 Navigation / KnowledgeAggregation 行为, 区分纯目录和知识领域
—	研究课题组地图	展望: 基于 graph.db 的 BelongTo / SameProject / CollaboratesWith 关系实现世界地图可视化
—	Metadata 判定原则	新增 (v2.2): 可作为 View 筛选条件的数据一律属于 metadata。跨层筛选由 View 引擎 join 查询, 不冗余写入
—	Event System & Transaction	新增 (v2.2): 事件驱动的事务架构。NeuronMoved 等事件通过 Event Bus → Listener 链驱动跨层更新, SQLite SAVEPOINT + 逆操作 + event_log.db 三层回滚保障

原始概念	最终定位	升级说明
—	Knowledge.md 五章结构	重构（v2.2）：统一为 总结 / 为什么 / 怎么做 / 思考 / 经验。Title/Tags/Status/Last Updated 等 metadata 字段移出 Knowledge.md，归属 metadata.json
—	event_log.db	新增（v2.2）：.neuro/persist/ 中新增事件日志数据库，记录每次事务操作，支持崩溃恢复时前向补执行或回滚

文档状态：PAD v2.2（产品架构文档）

核心记住一句话：

Everything is a Neuron.

Neuro 不是一个知识管理工具。

Neuro 是一个 AI 原生的知识培育系统（Knowledge Cultivation System）。

下一个要解决的问题：如何让 AI 在对的时候、用对的方式、干对的事，而不需要用户成为提示词工程师。